



UNIVERSIDAD DE MÁLAGA



Grado en Ingeniería Informática

Desarrollo de un sistema inteligente explicable basado en Graph-RAG y Análisis Formal de Conceptos

Development of an explainable intelligent system based on Graph-
RAG and Formal Concept Analysis

Realizado por
Miguel Ángel Caballero Serrano

Tutorizado por
Ángel Mora Bonilla

Departamento
Matemática Aplicada

MÁLAGA, junio 2026



UNIVERSIDAD
DE MÁLAGA



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA
INFORMÁTICA
GRADUADO EN INGENIERÍA INFORMÁTICA

**Desarrollo de un sistema inteligente explicable basado
en Graph-RAG y Análisis Formal de Conceptos**

**Development of an explainable intelligent system based on
Graph-RAG and Formal Concept Analysis**

Realizado por
Miguel Ángel Caballero Serrano

Tutorizado por
Ángel Mora Bonilla

Departamento
Matemática Aplicada

UNIVERSIDAD DE MÁLAGA
MÁLAGA, JUNIO DE 2026

Fecha defensa: julio de 2025

Resumen

El presente Trabajo Fin de Grado se enmarca dentro del uso de los Modelos de Lenguaje de Gran Tamaño (LLMs) y los sistemas de recuperación de información mediante grafos (Graph-RAG). Actualmente, los modelos de lenguaje cometen errores (conocidos como alucinaciones) y no explican cómo llegan a sus conclusiones. Aunque el uso de Graph-RAG mejora la búsqueda de contexto, carece del rigor matemático necesario para garantizar que las respuestas sean totalmente ciertas.

El objetivo principal de este trabajo es diseñar una arquitectura que integre el Análisis Formal de Conceptos (FCA). De este modo, FCA actúa como una guía lógica y matemática para controlar cómo el modelo genera el texto.

Para el desarrollo del proyecto se ha implementado una aplicación interactiva en el lenguaje R utilizando Shiny. En ella, se usa el paquete fcaR para extraer el conocimiento de los datos en forma de retículos de conceptos y reglas de implicación lógica. Además, este sistema se conecta con un modelo de lenguaje que se ejecuta en el propio ordenador del usuario (en local) usando el paquete ellmer y la herramienta Ollama, lo que asegura la privacidad de los datos.

Este trabajo demuestra que inyectar reglas lógicas al modelo reduce de forma importante las alucinaciones frente a los sistemas convencionales. El usuario dispone de diagramas visuales jerárquicos que le permiten ver y entender los pasos lógicos que el sistema ha seguido para dar una respuesta. Así, se consigue un sistema de inteligencia artificial fiable y fácil de explicar.

Palabras clave: Análisis de Conceptos Formales, Graph-RAG, Modelos de Lenguaje de Gran Tamaño, Interfaz de Usuario, Inteligencia Artificial Fiable, Lenguaje de Programación R.

Abstract

This Final Year Dissertation is framed within the use of Large Language Models (LLMs) and graph-based retrieval-augmented generation systems (Graph-RAG). Currently, language models make errors (known as hallucinations) and do not explain how they reach their conclusions. Although the use of Graph-RAG improves context retrieval, it lacks the mathematical rigor necessary to guarantee that the answers are entirely accurate.

The main objective of this work is to design an architecture that integrates Formal Concept Analysis (FCA). In this way, FCA acts as a logical and mathematical guide to control how the model generates text.

For the development of the project, an interactive application has been implemented in the R programming language using Shiny. In it, the `fcaR` package is used to extract knowledge from the data in the form of concept lattices and logical implication rules. Furthermore, this system connects with a language model that runs on the user's own computer (locally) using the `ellmer` package and the Ollama tool, which ensures data privacy.

This work demonstrates that injecting logical rules into the model significantly reduces hallucinations compared to conventional systems. The user is provided with hierarchical visual diagrams that allow them to see and understand the logical steps the system has followed to provide an answer. Thus, a trustworthy and easy-to-explain artificial intelligence system is achieved.

Keywords: Formal Concept Analysis, Graph-RAG, Large Language Models, User Interface, Trustworthy Artificial Intelligence, R Programming Language.

Índice

Resumen.....	2
Abstract.....	3
Índice	5
Índice de Figuras.....	7
Introducción.....	9
1.1. Motivación	9
1.2. Objetivos.....	10
1.3. Estructura de la memoria.....	11
Metodología.....	13
2.1. Requisitos del sistema.....	13
2.1.1. Requisitos Funcionales.....	13
2.1.2. Requisitos No Funcionales.....	14
2.2. Diseño.....	14
2.2.1. Casos de uso.....	15
2.3. Implementación.....	16
2.4. Documentación	16
Tecnologías usadas.....	17
3.1. Entorno de Desarrollo y Programación.....	17
3.2. Arquitectura Base y Lógica Matemática.....	17
3.3. Inteligencia Artificial y Privacidad.....	18
3.4. Interfaz de Usuario y Explicabilidad Visual.....	18
3.5. Evaluación de Resultados.....	18
3.6. Herramientas de Gestión y Documentación.....	19
Marco teórico.....	21
4.1. RAG.....	21
4.1.1. Ciclo.....	22
4.1.2. Limitaciones en entornos prácticos.....	22
4.2. Graph-RAG	23
4.2.1. Categorías según las estructuras de grafos.....	23
4.2.2. Beneficios ante sistemas basados en bases de datos vectoriales.....	24
4.3. RAG Tradicional vs. Graph-RAG	24
4.4. FCA.....	25
4.4.1. Conceptos previos.....	25
4.4.2. Definiciones.....	25
4.4.3. Ejemplo sencillo con el paquete fcaR.....	26
FCA en Graph-RAG.....	29
5.1. Síntesis neuro-simbólica	29
5.2. El LLM como extractor y redactor	30

5.3.	El FCA como autoridad lógica.....	31
5.4.	Explicabilidad y trazabilidad.....	32
5.5.	Caso ilustrativo.....	32
Desarrollo aplicación Shiny.....		35
6.1.	Estructura del proyecto.....	35
6.2.	Motor de razonamiento.....	36
6.2.1.	Extracción del conocimiento.....	36
6.2.2.	Núcleo simbólico: Análisis de Conceptos Formales.....	37
6.2.3.	Recuperación mediante grafo (Graph-RAG).....	37
6.2.4.	Interpretación de la consulta.....	38
6.2.5.	Orquestador híbrido.....	38
6.2.6.	Redacción y verificación con el LLM.....	39
6.2.7.	Ejecución asíncrona.....	39
6.3.	Interfaz.....	39
6.3.1.	Entrada de conocimiento.....	40
6.3.2.	Interacción con el asistente.....	41
6.3.3.	Visualización.....	41
6.3.4.	Otros elementos transversales.....	45
6.4.	Caso de uso: consulta clínica.....	46
Validación y pruebas.....		49
Conclusiones y líneas futuras.....		49
Referencias.....		50
Apéndice A.....		55
Manual de Usuario.....		55

Índice de Figuras

Figura 1 Diagrama UML.....	15
Figura 2 Gráfica Coste-Rendimiento LLMs.....	21
Figura 3 Comparación entre RAG Y Graph-RAG.....	24
Figura 4 Captura de la traza real del caso ilustrativo tal como la muestra la aplicación.....	33
Figura 5 Resolución del modelo de lenguaje.....	33
Figura 6 Vista general de la aplicación al iniciarse.....	40
Figura 7 Vista de la aplicación tras la extracción de reglas.....	41
Figura 8 Grafo de conocimiento sobre un nodo concreto y sus vecinos.....	42
Figura 9 Retículo de conceptos en modo personalizado.....	43
Figura 10 Reglas FCA.....	43
Figura 11 Datos - Atributos más frecuentes.....	44
Figura 12 Datos - Tripletas.....	44
Figura 13 Datos - Nodos por tipo y tamaño de las reglas.....	44
Figura 14 Evaluación - Comparativa de enfoques.....	45
Figura 15 Consulta sobre un paciente, con la traza y enfoque del grafo.....	46
Figura 16 Consulta hipotética - Trazabilidad lógica.....	47
Figura 17 Consulta hipotética - Resolución.....	47
Figura 18 Consulta inversa.....	48

1

Introducción

1.1. Motivación

En los últimos años, los Modelos de Lenguaje de Gran Tamaño (LLMs) se han convertido en herramientas muy populares gracias a su gran capacidad para procesar texto y responder preguntas de forma natural. Sin embargo, su funcionamiento interno se basa en la predicción probabilística, en lugar de en el razonamiento deductivo sobre hechos reales, es decir, intentan adivinar cuál es la siguiente palabra más adecuada en lugar de razonar sobre hechos reales. Esta característica provoca uno de sus mayores defectos: la invención de información o alucinaciones, junto con la incapacidad de explicarle al usuario cómo han llegado a esa conclusión. Siendo esto fundamental en sectores donde la precisión es vital, como la medicina, las finanzas o el derecho.

Para mitigar este problema, se desarrollaron los sistemas RAG (Generación Aumentada por Recuperación), los cuales buscan información en bases de conocimientos externas y se la entregan al modelo para que base sus respuestas en datos reales. Dado que el RAG tradicional a menudo perdía el contexto general, se evolucionó hacia el Graph-RAG, una técnica que organiza la información en forma de grafo (una red que conecta conceptos y sus relaciones) para mejorar la comprensión global del texto. No obstante, aunque el Graph-RAG mejora la calidad del contexto, sigue presentando una limitación: conecta información de forma semántica y asociativa, pero carece de un rigor matemático que garantice que las conexiones conformen una ruta de inferencia determinista y libre de contradicciones.

Para cubrir esta carencia, se recurre al Análisis Formal de Conceptos (FCA), una teoría matemática basada en la teoría de retículos y el orden. El FCA es capaz de analizar un conjunto de

datos, agrupar los objetos que comparten características comunes, (conceptos formales) y, lo más importante, extraer reglas fijas y absolutas entre ellos, conocidas como implicaciones lógicas.

La motivación central de este trabajo nace de la necesidad de integrar lo mejor de ambos enfoques. El objetivo es utilizar la estructura del Análisis Formal de Conceptos para que actúe como una autoridad lógica sobre el modelo de lenguaje. De este modo, en lugar de permitir que la IA asocie ideas libremente con el riesgo de equivocarse, se le imponen unas reglas matemáticas estrictas extraídas de los propios conjuntos de datos.

El resultado que persigue esta integración es transformar una herramienta probabilística y opaca en un sistema de IA fiable y transparente. Al obligar al modelo a seguir las reglas del FCA, no solo se consigue reducir de forma drástica la posibilidad de generar alucinaciones, sino que se logra que el sistema sea totalmente explicable. Así, cualquier usuario podrá ver la ruta lógica exacta que el sistema ha seguido para resolver su consulta paso a paso.

1.2. Objetivos

El objetivo principal es el diseño y desarrollo de un sistema de IA explicable que integre la arquitectura Graph-RAG con FCA. El propósito fundamental es mitigar de forma controlada las alucinaciones de los LLMs y garantizar que todas las respuestas generadas cuenten con una trazabilidad lógica y matemática absoluta.

Para estructurar y alcanzar esta meta, el proyecto se desglosa en los siguientes cuatro objetivos específicos:

- **Investigación del estado del arte de las arquitecturas RAG y Graph-RAG:** Este objetivo inicial se centra en analizar la evolución de los sistemas de recuperación de información para entender sus capacidades y fallos actuales. Se estudiarán las limitaciones del RAG tradicional, el cual recupera fragmentos de texto basándose únicamente en la similitud estadística, lo que a menudo provoca la pérdida del contexto general y dificultades para realizar razonamientos complejos. Posteriormente, se investigará a fondo el paradigma Graph-RAG, que organiza el conocimiento externo en forma de grafos. Se analizará cómo esta estructura permite a la IA conectar información dispersa y mejorar la comprensión global del texto.
- **Investigación sobre la integración de FCA en los grafos de Graph-RAG:** Aunque Graph-RAG mejora la búsqueda de contexto, sus grafos conectan información de forma visual y

semántica, pero carecen de leyes matemáticas estrictas que aseguren que esas conexiones sean lógicamente ciertas. El objetivo es investigar cómo puede aportar el rigor matemático que le falta al sistema. Se analizará el proceso para transformar los datos del grafo en contextos formales que permitan extraer jerarquías exactas (retículos de conceptos) y reglas fijas (bases de implicaciones lógicas). Estas reglas actuarán como una autoridad que dictará al modelo de lenguaje qué es verdad y qué no, bloqueando posibles alucinaciones lógicas.

- **Desarrollo de una aplicación en Shiny como motor de inferencia neuro-simbólico:** Consistirá en construir una herramienta de software que conecte la base matemática con el modelo de lenguaje. Los hitos técnicos incluyen: la utilización del paquete matemático fcaR en el entorno de programación R para extraer automáticamente los retículos de conceptos y las reglas lógicas a partir de los datos; la integración con modelos de lenguaje ejecutados en local mediante el servidor Ollama y el paquete ellmer, asegurando la total privacidad de la información; y el desarrollo de la interfaz de usuario en Shiny, que incluirá herramientas visuales para mostrar de forma gráfica e interactiva el razonamiento exacto de la IA.
- **Validación de la arquitectura y análisis de resultados:** Se ha diseñado un protocolo de evaluación propio que compara varias configuraciones. Desde el LLM aislado y los sistemas RAG y Graph-RAG convencionales hasta la arquitectura completa Graph-RAG+FCA. Todo ello sobre una batería de preguntas con respuesta de referencia donde se cuantifican la exactitud, la precisión, el recall y, especialmente, la tasa de alucinación, verificando que las configuraciones deterministas no inventan información. Asimismo, se comprueba la capacidad del sistema para emitir una justificación auditable de cada respuesta, demostrando que se ha transformado un modelo probabilístico opaco en una IA fiable y transparente.

1.3. Estructura de la memoria

El presente documento se divide en las siguientes secciones:

- **Resumen / Abstract:** Se realiza una síntesis breve e independiente que engloba el problema, los objetivos y los resultados obtenidos en el proyecto.
- **Introducción:** Se expone el contexto del proyecto, justificando la motivación para mitigar las alucinaciones de la IA, se definen los objetivos principales y específicos a alcanzar y, por último, se detallan las secciones de la memoria.

- **Metodología y tecnologías usadas:** Se describe el proceso de trabajo seguido y las herramientas como lenguajes y librerías utilizadas junto con los componentes adicionales necesarios.
- **Marco Teórico (Estado del Arte):** Se establecen los fundamentos teóricos necesarios, analizando RAG, Graph-RAG y la teoría del Análisis Formal de Conceptos.
- **FCA en Graph-RAG:** Se recoge cómo integrar el conocimiento de FCA en los grafos de Graph-RAG.
- **Desarrollo de la Aplicación Shiny:** Se presenta la implementación técnica de la interfaz visual con la que interactuará el usuario y cómo se muestran los diagramas lógicos para la explicabilidad.
- **Validación y pruebas:** Demostración final de que el sistema desarrollado realmente mitiga los errores frente a sistemas Graph-RAG convencionales.
- **Conclusiones y líneas futuras:** Se realiza un balance sobre si se han cumplido los objetivos y propone nuevas ideas o mejoras que se podrían investigar en el futuro.
- **Referencias:** Se recogen las diferentes referencias bibliográficas que se consultaron durante el desarrollo.
- **Apéndice:** Se detallan el material extra y el Manual de Usuario.

2

Metodología

Para el desarrollo de este trabajo se ha adoptado una metodología Agile, basada en el desarrollo iterativo e incremental. Esta metodología plantea la división del trabajo en fases o sprints, al término de cada una de las cuales se obtiene una parte funcional y evaluable del proyecto, lo que permite detectar y corregir errores de forma temprana, minimizando su propagación.

A diferencia del Modelo en Cascada, que sigue un enfoque lineal y secuencial en el que cada fase (requisitos, diseño, implementación, pruebas y despliegue) debe completarse antes de iniciar la siguiente, la metodología Agile ofrece mayor flexibilidad ante el cambio. Aplicar un enfoque en cascada en este trabajo supondría una limitación significativa, dado que tanto los requisitos como los obstáculos técnicos pueden evolucionar a lo largo del proceso, lo que obligaría a rediseñar el proyecto desde el inicio ante cualquier modificación.

2.1. Requisitos del sistema

Para garantizar el correcto funcionamiento de la arquitectura propuesta, se han establecido los siguientes requisitos funcionales y no funcionales.

2.1.1. Requisitos Funcionales

- **RF1. Carga y procesamiento de documentos:** El sistema debe permitir la subida de documentos, a partir de los cuales se extraerán los objetos, atributos y relaciones presentes en los textos.
- **RF2. Extracción de lógica matemática:** El sistema debe utilizar el paquete fcaR para analizar los datos extraídos y calcular automáticamente las agrupaciones y las reglas matemáticas fijas.

- **RF3. Interfaz de chat interactiva:** Se debe proporcionar un espacio interactivo donde se puedan realizar consultas sobre los documentos utilizando lenguaje natural.
- **RF4. Generación de respuestas guiadas:** Ante una consulta, el sistema debe buscar en su base matemática y enviar al modelo de lenguaje tanto el contexto como las reglas lógicas que el modelo debe obedecer.
- **RF5. Validación de errores:** Antes de mostrar la respuesta, el sistema debe comprobar lógicamente que el texto generado por la IA no contradice ninguna regla matemática del conjunto de datos.
- **RF6. Visualización del razonamiento:** Se deben generar gráficos interactivos mediante visNetwork para representar el camino lógico que ha seguido la IA para llegar a su conclusión.

2.1.2. Requisitos No Funcionales

- **RNF1. Privacidad total y seguridad:** El sistema debe funcionar de manera completamente local utilizando la herramienta Ollama, sin enviar datos a servidores externos.
- **RNF2. Fiabilidad y veracidad:** Se debe garantizar que las respuestas generadas sean ciertas desde un punto de vista matemático, minimizando la probabilidad de alucinaciones.
- **RNF3. Trazabilidad y auditoría:** Cada respuesta emitida por la IA debe ser transparente, permitiendo revisar los pasos exactos que tomó el sistema.
- **RNF4. Fluidez y tiempo de respuesta:** Se deben implementar procesos asíncronos en la interfaz Shiny para evitar que la aplicación se bloquee mientras el modelo de lenguaje procesa la respuesta.
- **RNF5. Usabilidad:** La aplicación debe presentar un diseño accesible, de modo que no se requieran conocimientos avanzados en programación o matemáticas para auditar las respuestas del modelo.

2.2. Diseño

Las herramientas que se han utilizado durante el transcurso del proyecto serán definidas a continuación, donde el lenguaje de programación principal sería R, dado que constituye el núcleo del ecosistema del paquete matemático fcaR y facilita la orquestación de modelos de lenguaje mediante la librería ellmer. Además, el entorno interactivo se desarrollaría utilizando el marco de trabajo Shiny,

pues proporciona la infraestructura necesaria para desplegar aplicaciones web directamente desde R.

En esta fase se definió, a nivel estético y funcional, cuáles serían los resultados esperados y la arquitectura visual del sistema. Para ello, se comenzó realizando varios bocetos o mockups de la aplicación utilizando herramientas de diseño vectorial colaborativo como Figma. A través de este proceso, se estructuró la interfaz en torno a los módulos críticos del sistema neuro-simbólico: un panel de ingesta de documentos, un entorno de chat conversacional y un área de explicabilidad visual. Esto se realizó para prever cuáles iban a ser las funcionalidades disponibles y cómo se iban a distribuir en la pantalla sin sobrecargar al usuario.

Además, otro paso fundamental en esta fase fue la elaboración de diagramas UML para visualizar la arquitectura del software.

2.2.1. Casos de uso

Un caso de uso constituye una representación gráfica de los requisitos funcionales, permitiendo observar cómo interactúa el usuario final con el sistema propuesto.

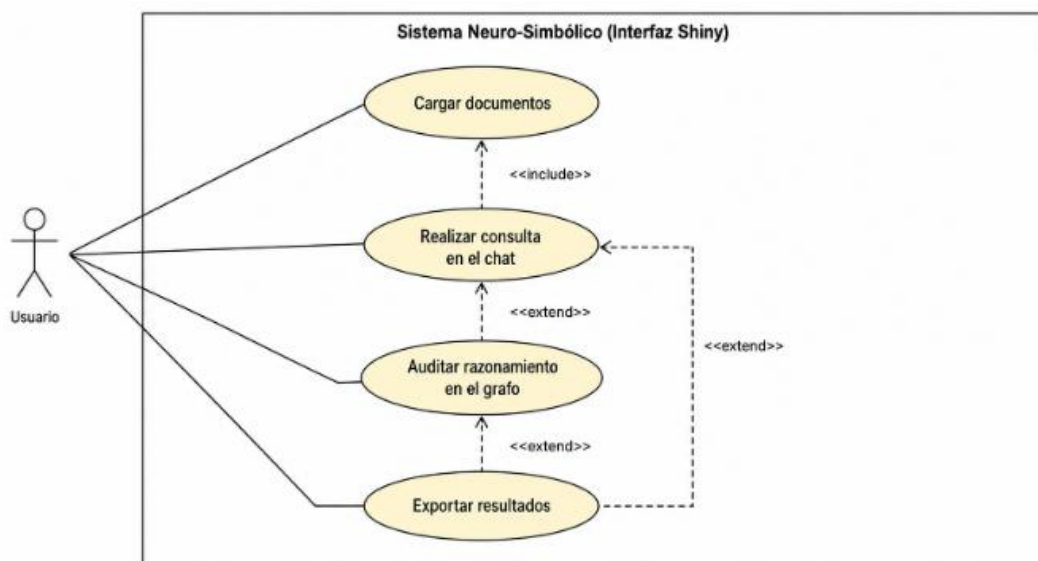


Figura 1 Diagrama UML

En el contexto de la arquitectura Graph-RAG + FCA desarrollada, el flujo de trabajo principal del usuario se articula en los siguientes pasos:

- **Carga y procesamiento de contexto:** Una vez proporcionados, el sistema procesa la información en segundo plano, extrayendo las entidades y delegando en fcaR el cálculo del retículo de conceptos y la base de implicaciones lógicas.

- **Interacción conversacional (Chat):** Una vez formalizado el contexto, el usuario puede introducir consultas en lenguaje natural, que desencadena el proceso de recuperación, donde el sistema inyecta las reglas matemáticas en el prompt y se comunica con el Modelo de Lenguaje para generar una respuesta.
- **Auditoría y Explicabilidad Visual:** Tras recibir la respuesta del asistente, el usuario puede navegar a una pestaña dedicada a la visualización. En ella, se renderiza de forma interactiva el Diagrama de Hasse, permitiendo al usuario ver la ruta que el sistema ha seguido para alcanzar su conclusión.
- **Exportación de resultados:** Por último, la aplicación ofrece funcionalidades de descarga en dos puntos del flujo. Durante la ingesta, permite exportar en formato CSV las tripletas de conocimiento extraídas del documento. En la pestaña de Evaluación, permite descargar un informe de resultados en HTML (que incluye el resumen por brazo, las gráficas comparativas y el detalle por pregunta) junto con los datos en bruto en CSV, de modo que las métricas y la comparativa entre arquitecturas pueden analizarse posteriormente sin necesidad de reprocesar los documentos.

2.3. Implementación

Para la elaboración del código, se han ido implementando de forma incremental las distintas funcionalidades disponibles. Se intercalaron fases de programación, de testeo y de revisión, teniendo en cuenta las funcionalidades dependientes de otras.

2.4. Documentación

A lo largo de la realización del proyecto, se ha ido documentando los cambios realizados de manera incremental.

3

Tecnologías usadas

El desarrollo se apoya en un conjunto de tecnologías de código abierto centradas principalmente en el ecosistema de programación R. La elección de estas herramientas persigue un doble objetivo: por un lado, permitir el procesamiento matemático complejo necesario para extraer la lógica de los datos; y por otro, garantizar que los Modelos de Lenguaje operen de forma local, interactiva y privada.

A continuación, se detallan las principales tecnologías empleadas en la arquitectura del sistema:

3.1. Entorno de Desarrollo y Programación

- **Lenguaje R:** Es el lenguaje de programación principal utilizado en el proyecto. Actúa como la columna vertebral que une el procesamiento de datos, la lógica matemática, la inteligencia artificial y la interfaz visual.
- **RStudio:** Es el entorno de desarrollo integrado (IDE) que se ha utilizado para escribir, organizar y probar todo el código escrito en el lenguaje R de una manera cómoda y eficiente.

3.2. Arquitectura Base y Lógica Matemática

- **Graph-RAG:** Es el paradigma o concepto de diseño principal del proyecto. A diferencia de los buscadores tradicionales, organiza la información en forma de grafo para que el sistema comprenda el contexto general de los datos.

- **lgraph**: construye el grafo dirigido y tipado que materializa el Graph-RAG, del que se recupera el conocimiento entidad-entidad.
- **fcaR**: Es un paquete matemático de R encargado del Análisis Formal de Conceptos, construye el contexto formal a partir de la matriz objeto-atributo y calcula los conceptos y la base de implicaciones de Duquenne-Guigues.
- **pdftools, jsonlite, dplyr y tidyr**: utilidades de ingesta y datos como lectura de PDF, parseo del JSON de extracción y construcción de la matriz del contexto formal.

3.3. Inteligencia Artificial y Privacidad

- **Ollama**: Es una aplicación que permite descargar y ejecutar LLMs directamente en el ordenador del usuario. Su uso garantiza que la información y los documentos procesados nunca viajen por internet, asegurando una privacidad total de los datos.
- **ellmer**: Es una librería de R que actúa como puente de comunicación. Permite que nuestro código en R envíe las preguntas del usuario y las reglas lógicas extraídas por fcaR al modelo de inteligencia artificial que está funcionando dentro de Ollama.
- **mirai**: ejecuta el LLM de manera asíncrona para que la interfaz no se bloquee mientras genera la respuesta.

3.4. Interfaz de Usuario y Explicabilidad Visual

- **Shiny**: Es una herramienta de R que permite construir aplicaciones web interactivas sin necesidad de programar en lenguajes web tradicionales. Se ha utilizado para diseñar la pantalla principal donde el usuario sube sus documentos y chatea con el sistema.
- **bslib**: define el tema, la maquetación y los componentes de la interfaz.
- **visNetwork**: Es un paquete de R especializado en dibujar redes interactivas. Se utiliza en la aplicación para mostrarle al usuario el grafo y las rutas lógicas (Diagramas de Hasse) que el sistema ha seguido, para entender por qué la IA dio una respuesta específica.
- **reactable**: Tablas interactivas (reglas, tripletas y resultados de evaluación).
- **plotly y ggplot2**: Tráficadas del cuadro de métricas y del informe de evaluación.

3.5. Evaluación de Resultados

- **Módulo de evaluación propio (R):** compara las distintas configuraciones (LLM solo, RAG, Graph-RAG y Graph-RAG+FCA) sobre una batería de preguntas con respuesta esperada, midiendo exactitud, precisión/recall/F1 y tasa de alucinación de forma determinista. Se valoró el uso de RAGAS, pero se descartó por basarse en un LLM-juez probabilístico, menos coherente con el objetivo de una medición reproducible.

3.6. Herramientas de Gestión y Documentación

- **GitHub:** Plataforma utilizada para alojar las distintas versiones del código fuente a lo largo del desarrollo. Ha permitido tener un control de cambios seguro, organizando el avance del software paso a paso.
- **Microsoft Word:** Procesador de textos utilizado para la elaboración de la memoria escrita de este Trabajo de Fin de Grado.

Marco teórico

4.1. RAG

Retrieval-Augmented Generation (Generación Aumentada por Recuperación) surge como solución a los altos costes computacionales y la complejidad técnica que supone entrenar o ajustar Modelos de Lenguaje de Gran Tamaño desde cero para dominios específicos. Aquí entra RAG, que es una técnica para mejorar la precisión y fiabilidad de los modelos generativos de IA con datos obtenidos de fuentes externas. En la siguiente gráfica se observa que se encuentra en un punto intermedio entre obtener un gran desempeño en cuanto a rendimiento y precisión sin que suponga gran coste de implementación.



Figura 2 Gráfica Coste-Rendimiento LLMs

4.1.1. Ciclo

El ciclo de funcionamiento de un sistema RAG tradicional se estructura en tres fases fundamentales:

1. Preparación e Indexación del Conocimiento Para que la información pueda ser consultada, primero debe ser agregada a una base de datos de la siguiente manera:

- 1. Fragmentación (Chunking):** Los documentos originales se dividen en fragmentos de texto más pequeños y manejables.
- 2. Vectorización (Embeddings):** Cada fragmento de texto se procesa mediante un modelo de representación para convertirlo en un embedding, representaciones vectoriales densas en un espacio matemático de alta dimensionalidad que capturan el significado semántico del texto.
- 3. Almacenamiento:** Estos vectores se almacenan en una Base de Datos Vectorial, sirviendo como índice de conocimiento.

2. Recuperación del Contexto (Retrieval) Cuando el usuario realiza una consulta o Query en lenguaje natural, en lugar de enviarla directamente al LLM para que responda de memoria, el sistema realiza una búsqueda de información, se transforma en un embedding; se realiza una búsqueda matemática para encontrar los fragmentos de documento en la base de datos cuyos vectores sean más similares al vector de la pregunta; y el sistema recupera los fragmentos de texto más relevantes.

3. Integración y Generación En la fase final, los documentos recuperados se empaquetan en un bloque denominado contexto, que se adjunta a la consulta original del usuario y se envía al LLM. El modelo genera una respuesta fundamentada estrictamente en los datos proporcionados.

En definitiva, este proceso favorece que la información devuelta por el LLM esté basada en el conocimiento propio y actualizado del usuario, mitigando las alucinaciones y permitiendo referenciar exactamente qué documentos dieron origen a la respuesta.

4.1.2. Limitaciones en entornos prácticos.

- **Dificultad en la comprensión de consultas complejas dentro de dominios especializados**, ya que los métodos tradicionales de recuperación no logran abarcar el razonamiento de múltiples pasos ni captar los matices semánticos profundos.

- **La integración de conocimiento distribuido**, ya que la práctica necesaria de fragmentar los textos para su indexación sacrifica contexto esencial.
- **Las restricciones propias de los LLM**, concretamente la limitación de sus ventanas de contexto, obligan a truncar la información, destruyendo el flujo lógico y las dependencias a largo plazo.
- **Eficiencia y escalabilidad**, puesto que la búsqueda en bases de conocimiento a gran escala incrementa los costes computacionales y la latencia, observando además una degradación en la precisión del sistema conforme la base de datos se expande.

4.2. Graph-RAG

Para superar estas deficiencias, emerge el paradigma de la Generación Aumentada por Recuperación basada en Grafos (Graph-RAG). A diferencia de los sistemas basados en bases de datos vectoriales planas, estructura la información mediante nodos (entidades) y aristas (relaciones semánticas). Este enfoque permite al LLM no solo recuperar datos locales, sino navegar a través de rutas conectadas, posibilitando una comprensión global del dominio y un razonamiento estructurado mucho más robusto.

4.2.1. Categorías según las estructuras de grafos

- **Basado en conocimiento:** Transforma texto no estructurado en Grafos de Conocimiento explícitos, modelando nodos y aristas.
- **Basado en índices:** Mantiene el texto original y emplea el grafo para conectar semánticamente fragmentos de texto.
- **Híbrido:** Combina las fortalezas de los grafos de conocimiento y de índices para abordar tareas de razonamiento de alta complejidad.

El funcionamiento de Graph-RAG comparte la estructura de tres etapas del RAG tradicional, pero adaptadas a una arquitectura de grafos. Primero, se estructura la información externa mediante grafos, ya sea creando representaciones explícitas del conocimiento o estableciendo redes de indexación semántica. A continuación, se emplean planificadores basados en grafos que no solo evalúan la similitud semántica, sino también la coherencia lógica, extrayendo subgrafos altamente relevantes para la consulta. Finalmente, el modelo fusiona el subgrafo recuperado con la consulta inicial del usuario.

4.2.2. Beneficios ante sistemas basados en bases de datos vectoriales

- **Representación avanzada del conocimiento:** Al capturar relaciones de múltiples saltos (multi-hop), el sistema comprende contextos jerárquicos y resuelve con mayor precisión consultas ambiguas o que requieren deducciones complejas.
- **Flexibilidad de integración:** Permite unificar en una sola estructura topológica diversas fuentes de información, abarcando datos estructurados, semiestructurados y texto no estructurado.
- **Eficiencia computacional y escalabilidad:** El recorrido de las bases de datos de grafos reduce la longitud del contexto necesario. Estudios evidencian que Graph-RAG genera respuestas utilizando entre un 26% y un 97% menos de tokens. Además, permite actualizar la base de conocimiento en tiempo real añadiendo nodos, sin necesidad de reindexar el corpus completo.
- **Interpretabilidad y trazabilidad:** La estructura de grafos hace que el proceso de razonamiento sea transparente y auditable. Es posible visualizar el camino lógico que el sistema ha seguido para generar una respuesta, una característica indispensable en sectores críticos como el legal, el financiero o el sanitario.

4.3. RAG Tradicional vs. Graph-RAG

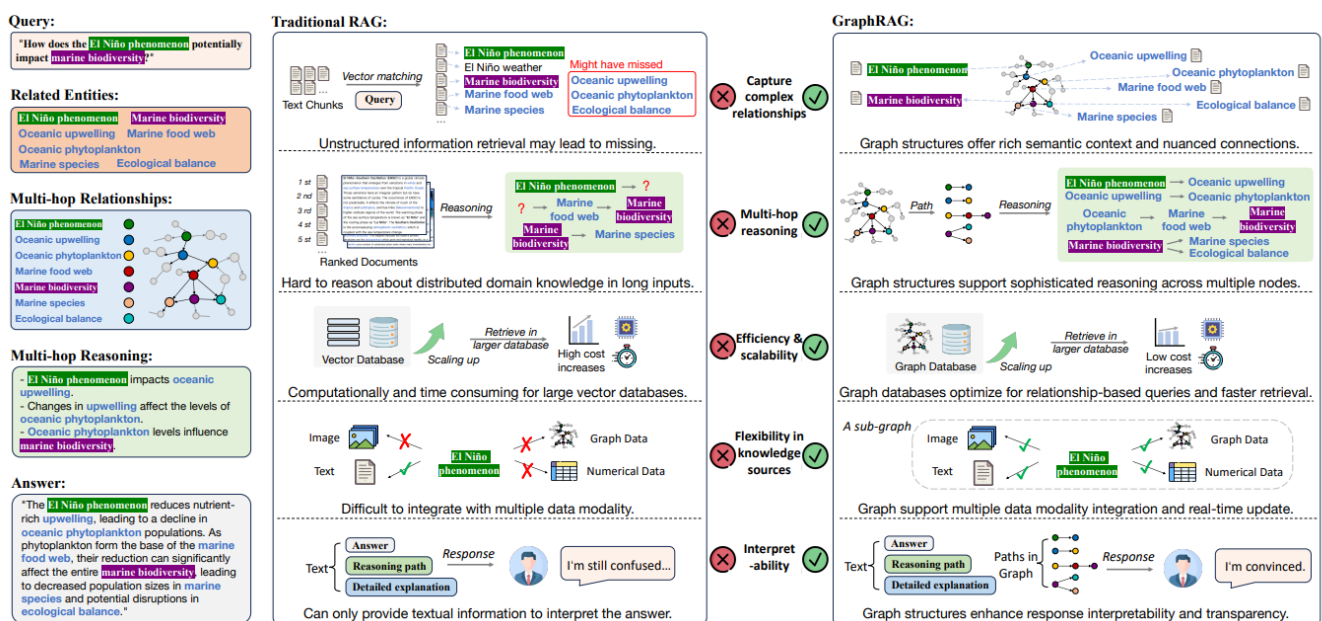


Figura 3 Comparación entre RAG Y Graph-RAG

La transición hacia el paradigma Graph-RAG surge para superar las limitaciones estructurales de la indexación vectorial pura. Mientras que el modelo RAG organiza el corpus en chunks sin conexión lógica, Graph-RAG emplea bases de datos orientadas a grafos. Esta arquitectura modela relaciones y jerarquías explícitas que preservan la integridad semántica global mediante enlaces topológicos. Con ello, se habilita un razonamiento de múltiples pasos (multi-hop) capaz de conectar conceptos dispersos para resolver consultas complejas. En consecuencia, Graph-RAG optimiza radicalmente la eficiencia al reducir el consumo de tokens, permite actualizaciones dinámicas sin necesidad de reindexar el corpus completo, y transforma la opacidad del filtrado vectorial en un proceso de inferencia auditable, trazable y visualizable.

4.4. FCA

4.4.1. Conceptos previos

El Análisis Formal de Conceptos fue introducido por Rudolf Wille en la década de 1980 basándose en la teoría de retículos de Birkhoff.

Nació con la voluntad de dotar de una finalidad aplicada y comunicativa a la teoría de retículos, sobre lo que el propio Wille afirmaba:

“El objetivo y el significado del FCA como teoría matemática sobre conceptos y sus jerarquías es apoyar la comunicación racional entre seres humanos mediante el desarrollo matemático de estructuras conceptuales apropiadas que se puedan manipular con la lógica”

4.4.2. Definiciones

1. Una **relación binaria** R se define como

$$R = \{ (a_1, a_2) : (a_1, a_2) \in A_1 \times A_2 \wedge R(a_1, a_2) = \text{Verdadero} \}$$

Siendo un conjunto formado por pares que están relacionados entre sí, mediante alguna propiedad determinada. Si se desea representar gráficamente la definición anterior, sería posible mediante un retículo, término que proviene de la forma de los diagramas de Hasse. Estos son los siguientes conceptos a introducir

2. Un **retículo** es una estructura algebraica con dos operaciones binarias y que forma un conjunto parcialmente ordenado, que cumple la propiedad de que toda pareja $a, b \in A$ tiene un único supremo $\sup(a, b) \in A$ y un único ínfimo $\inf(a, b) \in A$.

3. Un **diagrama de Hasse** es una representación gráfica de un conjunto parcialmente ordenado finito. Además de las definiciones anteriores, es interesante presentar la operación de derivación, pues nos servirá para trabajar con la relación P entre objetos y atributos.

Dados dos subconjuntos $X \subseteq O$ y $Y \subseteq A$, tenemos que:

$$X' = \{y \in A: \forall x \in X, P(x, y)\}$$

$$Y' = \{x \in O: \forall y \in Y, P(x, y)\}$$

Es decir, X' recoge los atributos comunes a todos los objetos en X e Y' los objetos que tienen todos los atributos en Y .

4. Un **contexto formal** $K := (G, M, I)$ está formado por dos conjuntos G y M y una relación I entre estos dos. Los elementos de G se denominan objetos y los de M atributos del contexto. Para expresar que un objeto g se encuentra en una relación I con un atributo m , se escribe gIm y se lee 'el objeto g posee el atributo m '
5. Un **concepto formal** del contexto (G, M, I) es una tupla (A, B) con $A \subseteq G$, $B \subseteq M$, $A' = B$ y $B' = A$. Se denomina a A como **extensión** y a B **intensión**. $B(G, M, I)$ constituye el conjunto de todos los conceptos de un contexto (G, M, I) .

La operación de **extensión** permite, dado un conjunto de atributos, computar qué objetos los poseen, mientras que la **intensión** nos dice, dado una serie de objetos, qué atributos poseen. Además, también es posible el cómputo de implicaciones, que constituyen una serie de reglas que se cumplen para distintos conjuntos de atributos.

6. Se dice que dos subconjuntos A y B proporcionan la **implicación** $A \rightarrow B$ si se cumple $A' \subseteq B'$, es decir, que A y B constituyen todos los objetos que tienen en común sus atributos, respectivamente

La solidez demostrada por las estructuras del FCA contrasta con la naturaleza probabilística de los LLMs. Mientras que los LLMs destacan en la comprensión del lenguaje y la generación de texto, carecen de mecanismos internos de verificación lógica. Esta complementariedad fundamenta el paradigma de la **IA Neuro-Simbólica**, el cual propone integrar el motor de razonamiento matemático del FCA como una capa de autoridad lógica sobre la recuperación de información estructurada (Graph-RAG). De este modo, las deficiencias de alucinación de las redes neuronales se restringen y verifican en tiempo real mediante las implicaciones deterministas extraídas de los datos.

4.4.3. Ejemplo sencillo con el paquete fcaR

Con el objetivo de asentar todos estos conceptos teóricos, resulta conveniente plantear un ***ejemplo práctico pedagógico*** que permita clarificar las ideas más complejas (en el capítulo de Desarrollo se aplicará el mismo procedimiento al ***dominio médico real*** de la aplicación).

Para ello, se emplea el paquete fcaR, el cual proporciona una implementación sencilla e intuitiva de los métodos clásicos usados en FCA, así como de los algoritmos desarrollados en las investigaciones de los tutores de este TFG.

El primer paso consiste en definir el conjunto de datos sobre el que se desarrollará el ejemplo. En este caso, el conjunto de objetos está compuesto por una selección de animales. Por su parte, el conjunto de atributos representa diversas características biológicas o de hábitat que estos poseen, tales como si son mamíferos, terrestres, acuáticos o si tienen la capacidad de volar.

	Mamífero	Ave	Terrestre	Acuático	Vuela
Gato	X		X		
Perro	X		X		
Delfín	X			X	
Ballena	X			X	
Gorrión		X	X		X
Pingüino		X		X	

Primero construimos el contexto formal mediante:

```
> fc <- FormalContext$new(datos_animales)
```

Esto nos proporciona la estructura básica a partir de la que trabajar. Para empezar, es posible computar los conceptos e implicaciones a través de los siguientes métodos, respectivamente:

```
> fc$find_concepts()
> print(fc$concepts)
A set of 10 concepts:
1: ({Gato, Perro, Delfín, Ballena, Gorrión, Pingüino}, {})
2: ({Delfín, Ballena, Pingüino}, {Acuatico})
3: ({Gato, Perro, Gorrión}, {Terrestre})
4: ({Gorrión, Pingüino}, {Ave})
5: ({Pingüino}, {Ave, Acuatico})
```

```
6: ({Gorrión}, {Ave, Terrestre, Vuela})
7: ({Gato, Perro, Delfín, Ballena}, {Mamifero})
8: ({Delfín, Ballena}, {Mamifero, Acuatico})
9: ({Gato, Perro}, {Mamifero, Terrestre})
10: ({}, {Mamifero, Ave, Terrestre, Acuatico, Vuela})
```

```
> fc$find_implications()
> print(fc$implications)
Implication set with 4 implications.
Rule 1: {Vuela} -> {Ave, Terrestre}
Rule 2: {Terrestre, Acuatico} -> {Mamifero, Ave, Vuela}
Rule 3: {Ave, Terrestre} -> {Vuela}
Rule 4: {Mamifero, Ave} -> {Terrestre, Acuatico, Vuela}
```

Existen dos funciones que permiten realizar operaciones de derivación: *intent()* y *extent()*. Estas operaciones sirven para obtener el conjunto de atributos que comparten una serie de objetos y viceversa.

Por ejemplo, dado el conjunto *Gato, Perro*, obtendríamos lo siguiente:

```
> S_objetos <- Set$new(attributes = fc$objects)
> S_objetos$assign(Gato = 1, Perro = 1)
> print(fc$intent(S_objetos))
{Mamifero, Terrestre}
```

Por otro lado, si se desea conocer qué animal es a su vez *mamífero y acuático*, obtendríamos los siguientes resultados:

```
> S_atributos <- Set$new(attributes = fc$attributes)
> S_atributos$assign(Mamifero = 1, Acuatico = 1)
> print(fc$extent(S_atributos))
{Delfín, Ballena}
```

5

FCA en Graph-RAG

La combinación del FCA con una arquitectura Graph-RAG da lugar al motor de inferencia neuro-simbólico que constituye el núcleo de este trabajo. Mientras que el capítulo anterior se presentó cada componente de forma aislada, en este se describe cómo se articulan entre sí: qué papel desempeña realmente el modelo de lenguaje, cómo el FCA aporta la autoridad lógica que garantiza la corrección, y de qué manera el grafo recupera el conocimiento que el contexto formal, por sí solo, no es capaz de capturar. El objetivo es que toda respuesta del sistema sea, a la vez, correcta por construcción y explicable paso a paso.

5.1. Síntesis neuro-simbólica

La arquitectura se fundamenta en la Teoría del Proceso Dual (Kahneman, 2011), que distingue dos modos de cognición complementarios: un **Sistema 1**, rápido, asociativo e intuitivo, y un **Sistema 2**, lento, deliberado y analítico. Trasladada a la inteligencia artificial, esta distinción permite asignar a cada tecnología el papel para el que resulta más apropiada.

En el sistema propuesto, el Sistema 1 lo desempeña el LLM, que aporta comprensión del lenguaje natural y reconocimiento de patrones, pero cuyo funcionamiento probabilístico lo hace propenso a la invención de información. El Sistema 2 lo desempeña el FCA, que aporta un razonamiento deductivo y determinista a partir de las reglas absolutas extraídas de los datos: la base de implicaciones de Duquenne-Guigues. Entre ambos se sitúa la capa de recuperación Graph-RAG, encargada de aportar

al razonamiento el conocimiento entidad-entidad estructurado que el contexto formal no representa de manera directa.

La clave de la integración es la subordinación del Sistema 1 al Sistema 2. El modelo de lenguaje no decide qué es verdad: el conocimiento que se entrega al usuario procede siempre del cierre lógico calculado por el FCA sobre la base de implicaciones, de modo que ninguna afirmación que llegue a la respuesta final puede contradecir las reglas del dominio. De esta forma, la naturaleza estocástica del LLM queda acotada por una capa de verificación matemática, y su capacidad generativa se limita a expresar en lenguaje fluido un resultado que ya ha sido deducido de manera fiable.

5.2. El LLM como extractor y redactor

Para que el motor lógico pueda ejercer su función, el rol del modelo de lenguaje dentro de la arquitectura queda deliberadamente acotado. El LLM interviene únicamente en tres tareas delimitadas, y en ninguna de ellas razona sobre la veracidad de los datos.

La primera es la **extracción de conocimiento** durante la ingesta de documentos. El modelo actúa como extractor de entidades y relaciones (**NER+RE**), procesando el texto en lenguaje natural y convirtiéndolo en tripletas de la forma **sujeto-relación-objeto**. Este proceso de autoformalización es el único punto en el que el LLM contribuye a construir la representación del dominio, y su salida se valida y normaliza antes de integrarse en el contexto formal.

La segunda es la **interpretación de la consulta**. Asimismo, esta interpretación se realiza, de forma prioritaria, mediante una capa determinista de procesamiento del lenguaje: un enrutador semántico que identifica en la pregunta los sujetos y atributos pertenecientes al vocabulario del dominio, reconoce la intención (consulta de un hecho, listado, filtro inverso o hipótesis) y detecta la categoría solicitada. El LLM interviene aquí solo como mecanismo de respaldo, cuando la capa determinista no logra resolver la entrada por la presencia de erratas o sinónimos, mapeando entonces la consulta al vocabulario reconocido. La construcción del contexto formal y la interpretación de las consultas son, por tanto, procesos deterministas; el LLM no sustituye a la lógica en ninguno de ellos.

La tercera es la **redacción de la respuesta**. Una vez que el motor simbólico ha deducido el resultado, el modelo retoma su capacidad generativa para sintetizar una respuesta natural y coherente. En esta fase su libertad está estrictamente restringida al conjunto de atributos deducido: la generación se ancla al veredicto lógico y, como se detalla en la sección siguiente, se verifica antes de mostrarse al usuario.

5.3. El FCA como autoridad lógica

El componente que garantiza la corrección del sistema es la base de implicaciones de Duquenne-Guigues, que constituye el conjunto mínimo de reglas lógicas que se cumplen para todos los objetos del contexto formal y que, por tanto, representan verdades inmutables del dominio. Sobre esta base, el razonamiento se realiza mediante cierre lógico (encadenamiento hacia delante): partiendo del conjunto de atributos de la consulta, se aplican de forma iterativa todas las implicaciones cuya premisa se cumple, incorporando sus consecuentes, hasta alcanzar un punto fijo en el que no es posible deducir ningún atributo nuevo. El resultado es un conjunto cerrado, lógicamente completo y libre de contradicciones.

Este razonamiento se complementa con la capa de recuperación Graph-RAG, que aporta el conocimiento entidad-entidad que el contexto formal no captura por co-ocurrencia. Dicho conocimiento se modela como relaciones tipadas en un grafo dirigido, pero solo una de esas relaciones, la relación "**implica**", propaga estado hacia el sujeto de la consulta. Se trata de una decisión de diseño deliberada: propagar otras relaciones (como `contraindicado_con`, `trata` o `pertenece_a`) introduciría en la respuesta atributos que no son ciertos para el sujeto, reintroduciendo precisamente el tipo de error que el sistema pretende evitar. Restringir la propagación a la relación `implica` mantiene la garantía de que todo atributo recuperado por el grafo es lógicamente atribuible al sujeto.

La autoridad lógica del FCA se ejerce en dos momentos:

- **Anclaje previo a la generación.** Antes de que el modelo de lenguaje redacte, el sistema calcula el cierre lógico de la consulta e incorpora el conocimiento recuperado por el grafo. El modelo no recibe la pregunta en abierto, sino un veredicto ya deducido; su tarea se reduce a expresarlo, no a resolverlo.
- **Verificación posterior.** La redacción producida por el modelo se contrasta contra ese veredicto determinista. Si el texto generado añade atributos no deducidos, niega algo que sí se deduce o contradice el resultado, la desviación se detecta y la redacción se sustituye por una versión determinista fiel al veredicto. De este modo, aunque el modelo cometa un error de expresión, este nunca llega al usuario.

Cabe precisar que el retículo de conceptos y su diagrama de Hasse no intervienen como motor de inferencia consulta a consulta; el razonamiento se apoya en el cierre sobre la base de implicaciones. El retículo se emplea, como se detalla a continuación, con fines de visualización y explicabilidad.

5.4. Explicabilidad y trazabilidad

La integración del FCA transforma un modelo probabilístico opaco en un sistema cuyas conclusiones son auditables. Dado que el cierre lógico es determinista y reproducible, cada respuesta puede acompañarse de la traza exacta del proceso de decisión, que el sistema presenta de forma estructurada en los siguientes pasos:

1. **Conjunto de partida:** los atributos que el sujeto (o la hipótesis planteada) tiene inicialmente.
2. **Recuperación Graph-RAG:** las aristas implica del grafo que se activan a partir de ese conjunto, mostradas explícitamente como relaciones entidad-entidad.
3. **Reglas FCA aplicadas:** las implicaciones de la base de Duquenne-Guigues que se disparan, enumeradas pasada a pasada, indicando en cada una qué atributos añade.
4. **Resultado deducido:** el conjunto cerrado final, agrupado por categoría clínica para facilitar su lectura.
5. **Resolución:** el veredicto en lenguaje natural, coherente con los pasos anteriores.

Esta traza permite a un usuario o auditor reconstruir el camino lógico completo, comprobar qué regla justifica cada conclusión y verificar que la respuesta se fundamenta exclusivamente en el conocimiento presente en los datos. La explicabilidad deja así de ser una aproximación a posteriori y pasa a ser una propiedad intrínseca del propio mecanismo de inferencia.

5.5. Caso ilustrativo

Para concretar el funcionamiento del paradigma se plantea un ejemplo del dominio clínico sobre el que opera la aplicación. Supóngase la consulta hipotética: "**Si un paciente tiene Asma, ¿qué riesgos tendría?**".

En una arquitectura Graph-RAG convencional, la respuesta dependería de que el modelo de lenguaje encontrara y conectara la información pertinente por similitud semántica, con el riesgo de omitir relaciones no explícitas o de completar el vacío con una suposición. En la arquitectura Graph-RAG + FCA, en cambio, el proceso es determinista y verificable.

El conjunto de partida es Asma. La capa de recuperación Graph-RAG localiza en el grafo la arista **Asma --implica--> Riesgo_Broncoespasmo** y la incorpora al razonamiento; a continuación, el FCA cierra el conjunto aplicando las implicaciones pertinentes de la base de Duquenne-Guigues. El sistema deduce así, con certeza y no por asociación, que un paciente asmático adquiere el atributo Riesgo_Broncoespasmo, y presenta la traza correspondiente (conjunto de partida, arista implica recuperada, reglas aplicadas y resultado). El modelo de lenguaje se limita a redactar ese resultado ya validado.

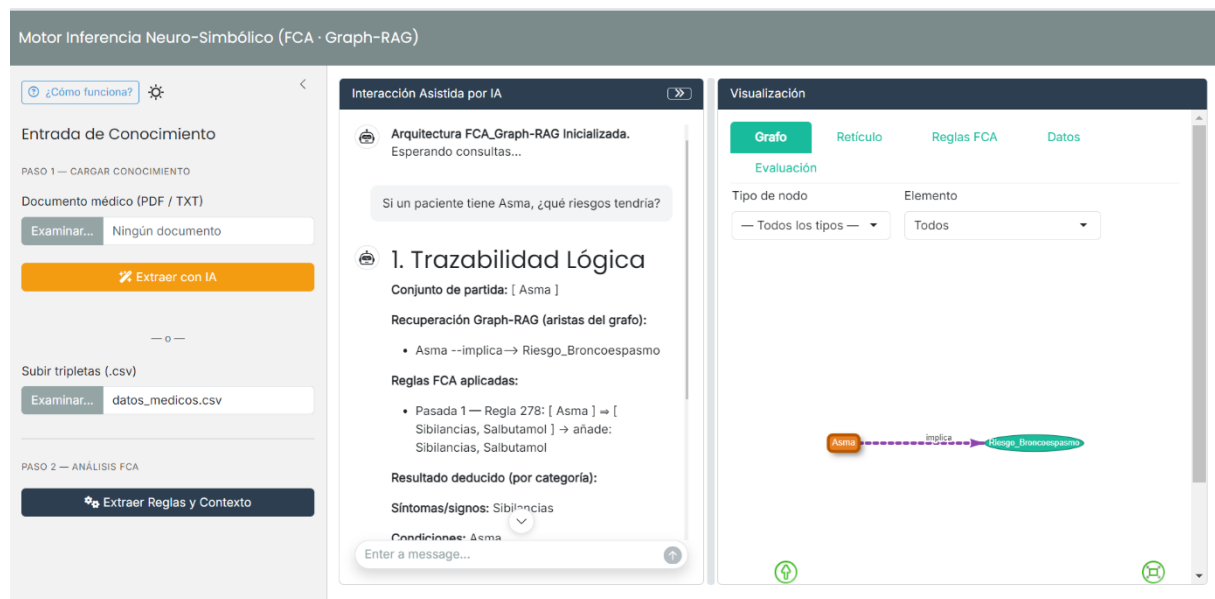


Figura 4 Captura de la traza real del caso ilustrativo tal como la muestra la aplicación

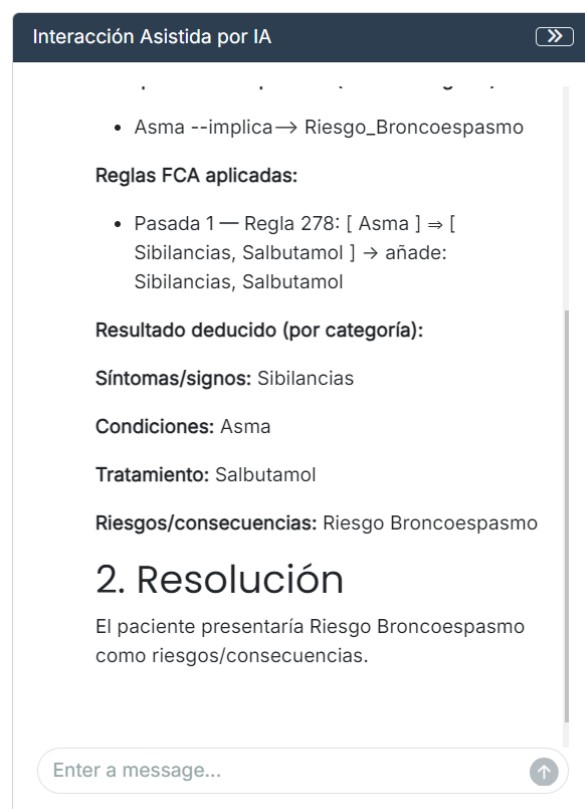


Figura 5 Resolución del modelo de lenguaje

El contraste es ilustrativo: la relación entre el asma y el riesgo de broncoespasmo constituye una regla del dominio que el grafo, por sí solo, no garantiza de forma determinista, pero que la

capa lógica del FCA sí extrae y verifica. La respuesta final es, en consecuencia, correcta, fundamentada y libre de alucinaciones, y su justificación queda registrada de manera reproducible.

6

Desarrollo aplicación Shiny

6.1. Estructura del proyecto

Debido a la complejidad del proyecto, se ha optado por una organización modular en lo que respecta a la estructura de ficheros.

En primer lugar, una aplicación Shiny contiene varios componentes fundamentales:

- **ui.R**: aloja la interfaz de usuario, es decir, la disposición de los paneles, los controles de entrada y las vistas de visualización.
- **server.R**: contiene la lógica reactiva de la aplicación, que conecta las acciones del usuario con el motor de razonamiento y actualiza las salidas.
- **app.R**: realiza la llamada a la función que arranca la aplicación, carga los módulos y prepara el proceso en segundo plano para el modelo de lenguaje.
- **global.R**: archivo con las declaraciones, librerías y funciones globales compartidas por toda la aplicación.

Para incrementar la legibilidad del código y separar responsabilidades, la lógica del sistema se ha dividido además en varios ficheros, cada uno encargado de una fase concreta del razonamiento:

- **extractor_tripletas.R**: gestiona la extracción del conocimiento; el modelo de lenguaje actúa como extractor y convierte un documento clínico en tripletas (sujeto, relación, objeto).
- **motor_simbolico.R**: construye el contexto formal, calcula el cierre lógico determinista y genera el grafo de conocimiento a partir de las tripletas.

- **recuperador_grafo.R:** implementa la capa Graph-RAG, que recupera conocimiento propagando por el grafo únicamente la relación de implicación.
- **enrutador_semantico.R:** interpreta la consulta en lenguaje natural, detectando los sujetos, los atributos, la relación consultada y la intención de la pregunta.
- **orquestador_hibrido.R:** coordina el enrutador, el motor simbólico y el grafo, y compone la respuesta con su traza lógica y su veredicto.
- **agente_llm.R:** gestiona la capa de redacción, donde el modelo de lenguaje traduce a prosa el veredicto determinista y se verifica que no lo contradiga.
- **graph_viz.R:** contiene la construcción y el estilo del grafo de conocimiento que se muestra en la interfaz.
- **presentacion.R:** reúne las utilidades de presentación, como el formato de las reglas o los indicadores del cuadro de mando.
- **evaluacion.R:** implementa la evaluación comparativa del sistema

6.2. Motor de razonamiento

El núcleo de la aplicación es un motor de razonamiento neuro-simbólico que combina tres capas con responsabilidades delimitadas. Un modelo de lenguaje se encarga de las tareas lingüísticas (extraer conocimiento del texto y redactar la respuesta final), mientras que el razonamiento propiamente dicho recae sobre el Análisis de Conceptos Formales (FCA), que actúa como un sistema deductivo determinista. Entre ambos, (Graph-RA rescata el conocimiento relacional que el razonamiento lógico, por sí solo, no alcanza.

Esta división responde a la tesis central del trabajo: el LLM **no razona**, sino que se limita a interpretar y a comunicar, de modo que la corrección de las conclusiones queda garantizada por la lógica y no por la plausibilidad estadística del modelo. En términos de la distinción entre pensamiento rápido e intuitivo (Sistema 1) y pensamiento lento y deliberativo (Sistema 2), el LLM desempeña el papel del Sistema 1 y el motor simbólico el del Sistema 2.

En las siguientes secciones se describe cada una de estas capas siguiendo el recorrido del conocimiento, desde la ingesta del documento hasta la elaboración de la respuesta.

6.2.1. Extracción del conocimiento

La primera fase transforma el texto clínico en conocimiento estructurado. Aquí el LLM actúa como extractor, realizando de forma conjunta el reconocimiento de entidades y la extracción de

relaciones: a partir de una frase, identifica las entidades relevantes y las relaciones que las vinculan, produciendo tripletas de la forma (sujeto, relación, objeto).

Para que la extracción sea fiable sobre documentos de cierta longitud, el texto se divide en fragmentos de tamaño acotado que se procesan de forma secuencial, informando del progreso fragmento a fragmento. El modelo se invoca con temperatura cero, lo que favorece un comportamiento determinista y reproducible. Su salida se solicita en formato estructurado y se interpreta mediante un analizador robusto, capaz de tolerar pequeñas variaciones en la respuesta del modelo sin descartar el resultado.

Un aspecto cuidado de esta fase es la **trazabilidad**: cada triplete conserva una referencia al fragmento del que procede, de modo que siempre es posible localizar en el documento original el origen de un dato extraído. Por último, las tripletas se depuran eliminando duplicados pero preservando el orden, y se consolidan en una tabla que constituye la entrada del resto del sistema.

6.2.2. Núcleo simbólico: Análisis de Conceptos Formales

Con las tripletas ya disponibles, el motor construye el **contexto formal**, que es la representación básica del FCA: una matriz binaria que relaciona un conjunto de objetos (pacientes, protocolos, entidades) con un conjunto de atributos, indicando qué atributos posee cada objeto. Sobre este contexto, el paquete fcaR calcula tanto el **retículo de conceptos** como la **base de implicaciones de Duquenne-Guigues**, que es un conjunto mínimo y completo de reglas lógicas capaz de generar todas las implicaciones válidas del contexto.

El componente central del razonamiento es el **cierre lógico**. Dada una entrada (como las condiciones que presenta un paciente), el cierre aplica de forma encadenada las reglas de la base hasta alcanzar el conjunto completo de atributos lógicamente implicados. Este proceso es puramente deductivo: solo produce conclusiones que se derivan necesariamente de las reglas, por lo que **no puede alucinar**. Es precisamente esta propiedad la que sustenta la fiabilidad del sistema, ya que traslada la responsabilidad de la corrección del modelo de lenguaje al aparato lógico.

A partir de las mismas tripletas, el motor construye además un **grafo dirigido y tipado** que servirá de soporte a la capa de recuperación descrita a continuación.

6.2.3. Recuperación mediante grafo (Graph-RAG)

El razonamiento por cierre lógico captura las dependencias entre atributos, pero existe conocimiento de naturaleza relacional, por ejemplo, que una condición conlleva un determinado

riesgo, la cual conviene recuperar explícitamente sobre el grafo. Esta es la función de la capa Graph-RAG.

El grafo representa cada entidad como un nodo y cada relación como una arista dirigida y etiquetada. La decisión de diseño más relevante de esta capa es que **solo se propaga un tipo de relación**: la relación de implicación. Es decir, al expandir el conocimiento a partir de las entidades de una consulta, el sistema recorre únicamente las aristas de implicación, recuperando así los riesgos y consecuencias asociados sin arrastrar relaciones de otra naturaleza que introducirían ruido. El resto de relaciones permanece en el grafo con fines descriptivos y de visualización, pero no interviene en la propagación.

De este modo, la capa de grafo complementa al cierre lógico: mientras el FCA deduce a partir de las reglas, el grafo aporta el conocimiento relacional de riesgo que aquellas no expresan, y ambos resultados se combinan en la fase de orquestación.

6.2.4. Interpretación de la consulta

Antes de razonar, el sistema debe entender qué se le pregunta. De ello se encarga el enrutador semántico, que constituye un componente de interpretación ligero (un Sistema 1 sencillo y determinista, sin recurrir al LLM salvo como último recurso). A partir del texto de la consulta, el enrutador identifica:

- Los **sujetos** mencionados (por ejemplo, un paciente o un fármaco concreto), reconociendo variantes de escritura.
- Los **atributos** presentes en la pregunta, con una detección tolerante a pequeñas erratas mediante comparación por ventanas de palabras.
- La **relación o categoría** consultada, que permite acotar la respuesta a un tipo concreto de información (síntomas, tratamiento, riesgos, etc.).
- La **intención** de la pregunta: si es directa o inversa, si es hipotética, o si pregunta por lo ausente.

Solo cuando la detección directa de un sujeto falla (por ejemplo, ante una errata importante) se recurre al LLM como apoyo de interpretación, siempre de forma acotada.

6.2.5. Orquestador híbrido

El orquestador es la pieza que coordina todo lo anterior y compone la respuesta. Recibe la interpretación del enrutador y, según la intención detectada, decide qué capas intervienen y en qué orden.

En una consulta factual sobre un sujeto, aplica el cierre lógico sobre sus atributos y agrupa el resultado por categorías clínicas. En una consulta inversa, recorre el contexto para localizar los objetos que poseen un atributo dado. En una consulta hipotética, combina el cierre lógico con la propagación por el grafo, de modo que a los atributos deducidos se suman los riesgos recuperados por la relación de implicación.

En todos los casos, el orquestador produce dos elementos: un **veredicto** determinista, que es el resultado del razonamiento, y una **traza lógica** que documenta el conjunto de partida y el cierre alcanzado. Esta separación es la que después permite a la interfaz mostrar de forma diferenciada lo que el motor ha deducido y lo que el modelo redacta.

6.2.6. Redacción y verificación con el LLM

Una vez obtenido el veredicto, el modelo de lenguaje vuelve a intervenir, esta vez como redactor. A partir del veredicto determinista se construye un mensaje que el modelo convierte en una respuesta en prosa natural, guiado por unas instrucciones que le prohíben añadir información o alterar las conclusiones.

Como salvaguarda adicional, la respuesta redactada se somete a una **verificación** que comprueba que no contradice el veredicto. Si se detecta una discrepancia, el sistema recurre a una redacción de respaldo generada de forma determinista a partir del propio veredicto. De este modo, incluso en la fase de comunicación, la última palabra la tiene la lógica y no el modelo.

6.2.7. Ejecución asíncrona

Dado que el modelo de lenguaje se ejecuta localmente e introduce latencias apreciables, la generación de la respuesta se realiza en un **proceso en segundo plano**, de forma que la interfaz no se bloquea mientras el modelo trabaja. Gracias a ello, la traza lógica se muestra al instante, y la resolución redactada se incorpora en cuanto está lista.

Para reducir la latencia se mantiene el modelo cargado en memoria entre consultas y se realiza un precalentamiento inicial, evitando el coste de arranque en la primera pregunta. El resultado es una experiencia fluida pese a apoyarse en un modelo de ejecución local.

6.3. Interfaz

La aplicación se organiza en tres zonas: a la izquierda, un panel de entrada de conocimiento; en el centro, el panel de interacción con el asistente; y a la derecha, el panel de visualización. La frontera entre los dos paneles principales es redimensionable, de modo que el usuario puede repartir el espacio

a su gusto, e incluso ocultar por completo la visualización para trabajar con la conversación a pantalla completa.

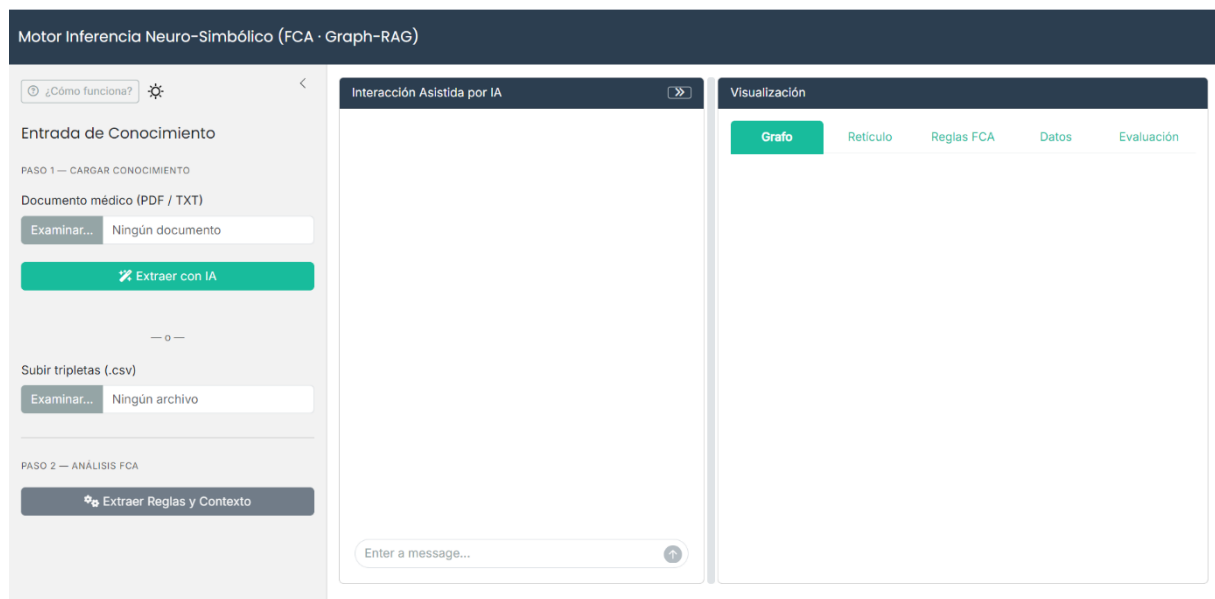


Figura 6 Vista general de la aplicación al iniciarse

6.3.1. Entrada de conocimiento

El primer paso, una vez se accede a la interfaz, es proporcionar el conocimiento con el que se va a trabajar. Para ello se ofrecen dos opciones:

- **Extraer de un documento con IA:** el usuario sube un documento clínico en formato PDF o TXT y, al pulsar el botón correspondiente, el modelo de lenguaje identifica y extrae las tripletas de conocimiento presentes en el texto.
- **Subir un CSV de tripletas:** alternativamente, si ya se dispone del conocimiento estructurado, es posible cargarlo directamente como un archivo CSV, sin pasar por la fase de extracción.

Ambas vías convergen en la misma representación interna, por lo que el resto de la aplicación es indiferente al origen del conocimiento. Tras una extracción con IA, la interfaz confirma el número de tripletas obtenidas y ofrece la posibilidad de descargar el CSV generado para su revisión.

Una vez cargados los datos, el segundo paso consiste en pulsar el botón de análisis, momento en el que el motor construye el contexto formal, calcula el retículo de conceptos y la base de implicaciones, y prepara el grafo de conocimiento. Este botón permanece deshabilitado hasta que existen datos cargados, lo que guía al usuario por el orden correcto de operaciones.

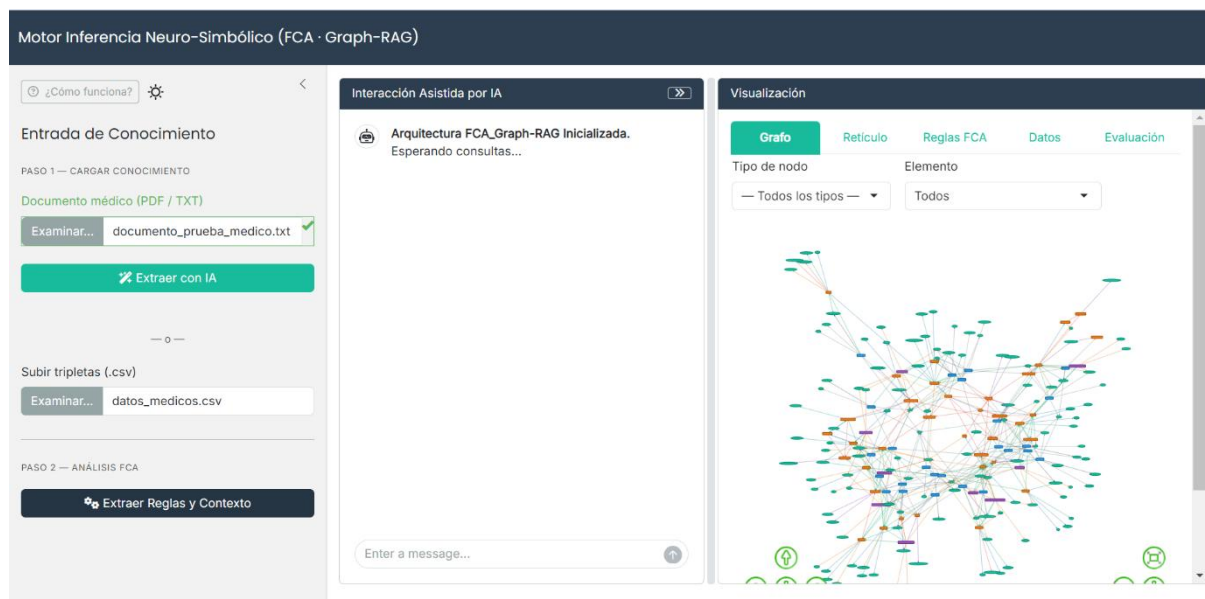


Figura 7 Vista de la aplicación tras la extracción de reglas

6.3.2. Interacción con el asistente

En el panel central se formulan las preguntas en lenguaje natural. Cada respuesta se compone de dos bloques claramente diferenciados:

1. **Traza lógica:** muestra el conjunto de partida, el cierre lógico deducido por el motor simbólico y el veredicto determinista.
2. **Resolución:** el texto que el LLM redacta a partir de ese veredicto, traducéndolo a texto natural comprensible.

Esta separación es deliberada, pues permite comprobar que la parte generativa se limita a expresar lo que la parte lógica ha establecido, sin añadir ni contradecir nada. Dado que la redacción corre a cargo del modelo local, la traza aparece de inmediato y la resolución se incorpora en cuanto el modelo termina, sin bloquear la interfaz.

6.3.3. Visualización

El panel derecho ofrece cinco vistas del conocimiento, organizadas en pestañas.

La primera es el **grafo de conocimiento**, donde los nodos se colorean según su tipo (paciente, protocolo, entidad y atributo) y las aristas se etiquetan según la relación que representan. Al hacer clic sobre un nodo, la vista se enfoca en él, ocultando los elementos no relacionados y reorganizando sus vecinos a su alrededor. Dos desplegable permiten filtrar por tipo de nodo o por elemento

concreto. Cuando se formula una consulta, el grafo aísla el sujeto y los atributos implicados y resalta las aristas de implicación recuperadas por la capa Graph-RAG, evidenciando la aportación del grafo al razonamiento.

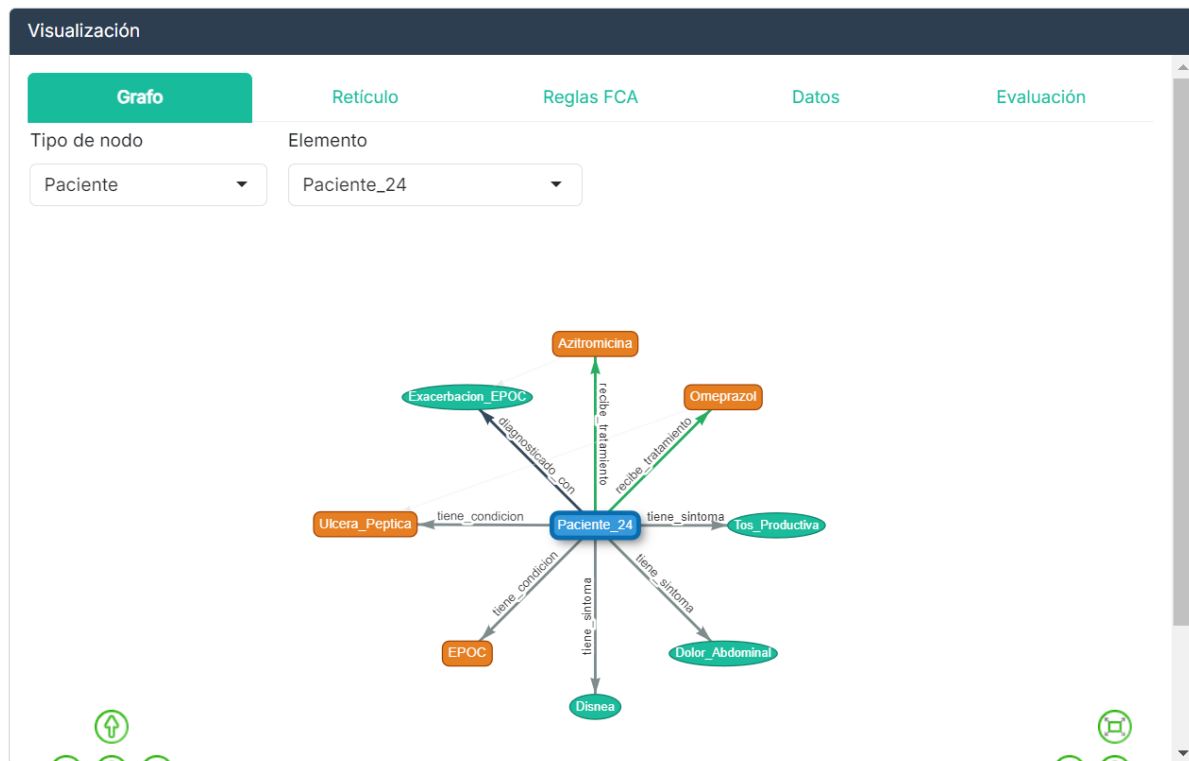


Figura 8 Grafo de conocimiento sobre un nodo concreto y sus vecinos

La segunda vista es el **retículo de conceptos** producido por el análisis de conceptos formales. Se ofrecen dos modos: uno interactivo, que construye un subretículo a partir de los atributos que el usuario seleccione, y otro que dibuja la estructura completa del retículo. El primero prioriza la legibilidad; el segundo, la visión global de la jerarquía.

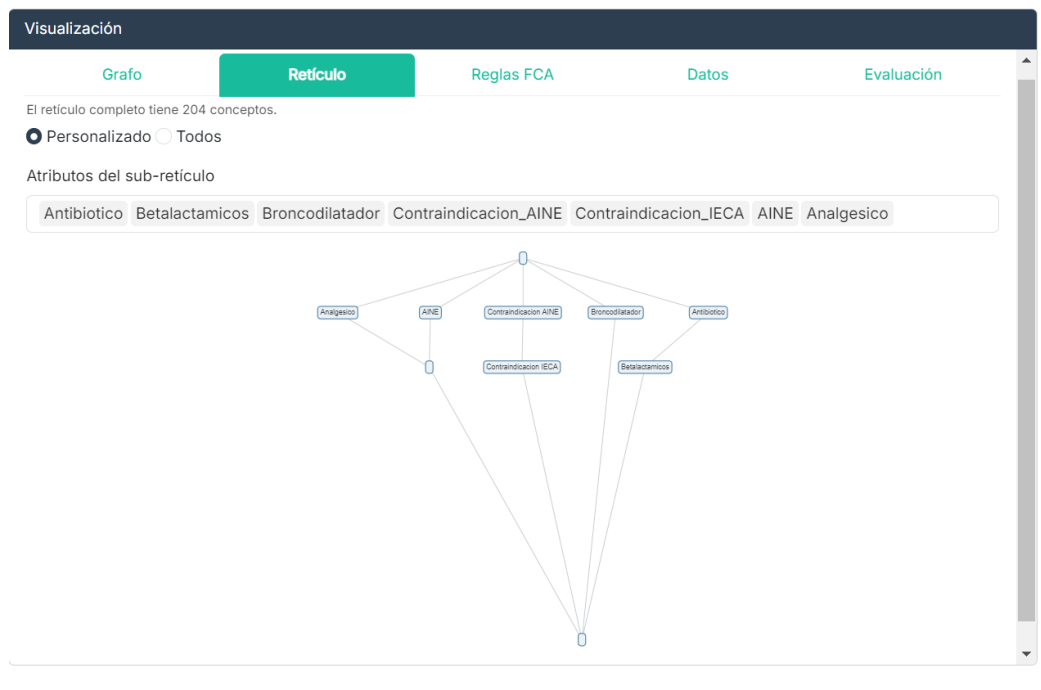


Figura 9 Retículo de conceptos en modo personalizado

La tercera vista muestra las **reglas de implicación** en una tabla interactiva, sobre la que se pueden aplicar filtros por atributo, por antecedente o consecuente, por tamaño de la regla y por rango. Esta vista hace tangible el carácter determinista del sistema, ya que cada respuesta puede rastrearse hasta las reglas que la sustentan.

Visualización

Grafo Retículo **Reglas FCA** Datos Evaluación

Total: 393 reglas. Filtra y elige el rango:

Buscar atributo

p.ej. Asma

Buscar en

☒ Cualquiera ☐ Antecedente ☐ Consecuente

Nº de atributos por lado:

Antecedente Consecuente

Máx.

Rango a mostrar:

Desde Hasta Regla concreta

Personalizado 1 10

Mostrando 1-10 de 393

Rule 1:	[Contraindicacion_Warfarina] $\Rightarrow$ [Contraindicacion_AINE, Contraindicacion_IECA]
Rule 2:	[Contraindicacion_IECA] $\Rightarrow$ [Contraindicacion_AINE, Contraindicacion_Warfarina]
Rule 3:	[Riesgo_Nefropatia] $\Rightarrow$ [Riesgo_Hipoglucemia]
Rule 4:	[Riesgo_Hipoglucemia] $\Rightarrow$ [Riesgo_Nefropatia]
Rule 5:	[Riesgo_Insuficiencia_Respiratoria] $\Rightarrow$ [Riesgo_Broncoespasmo]

Figura 10 Reglas FCA

La cuarta vista, **Datos**, reúne la visión cuantitativa del conocimiento: cuatro indicadores clave (conceptos, reglas, entidades y atributos), varias gráficas analíticas (atributos más frecuentes, nodos por tipo y tamaño de las reglas) y un explorador de las tripletas extraídas.



Figura 11 Datos – Atributos más frecuentes

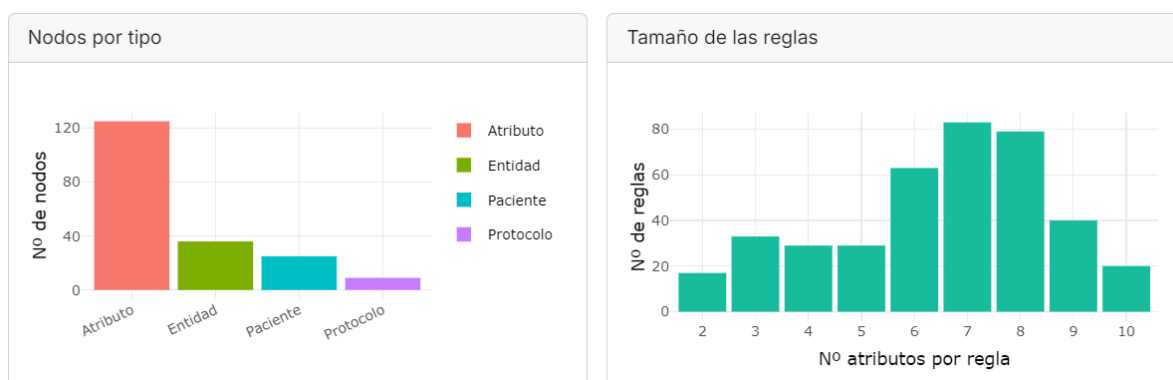


Figura 13 Datos – Nodos por tipo y tamaño de las reglas

Visualización

Métricas

Tripletas

Search

Sujeto	Relacion	Objeto
Paciente_01	tiene_sintoma	Fiebre
Paciente_01	tiene_sintoma	Tos_Productiva
Paciente_01	tiene_sintoma	Dificultad_Respiratoria
Paciente_01	tiene_sintoma	Dolor_Toracico
Paciente_01	tiene_condicion	Alergia_Penicilina
Paciente_01	diagnosticado_con	Neumonia_Bacteriana
Paciente_01	recibe_tratamiento	Azitromicina
Paciente_02	tiene_sintoma	Sibilancias
Paciente_02	tiene_sintoma	Disnea
Paciente_02	tiene_sintoma	Tos_Seca
Paciente_02	tiene_sintoma	Taquicardia
Paciente_02	tiene_condicion	Asma
Paciente_02	diagnosticado_con	Crisis_Asmatica

Figura 12 Datos – Tripletas

La quinta vista, **Evaluación**, integra en la propia aplicación la comparativa entre los distintos enfoques del sistema y permite exportar los resultados como informe HTML o como datos en CSV. Su fundamentación metodológica se aborda en el capítulo de validación.

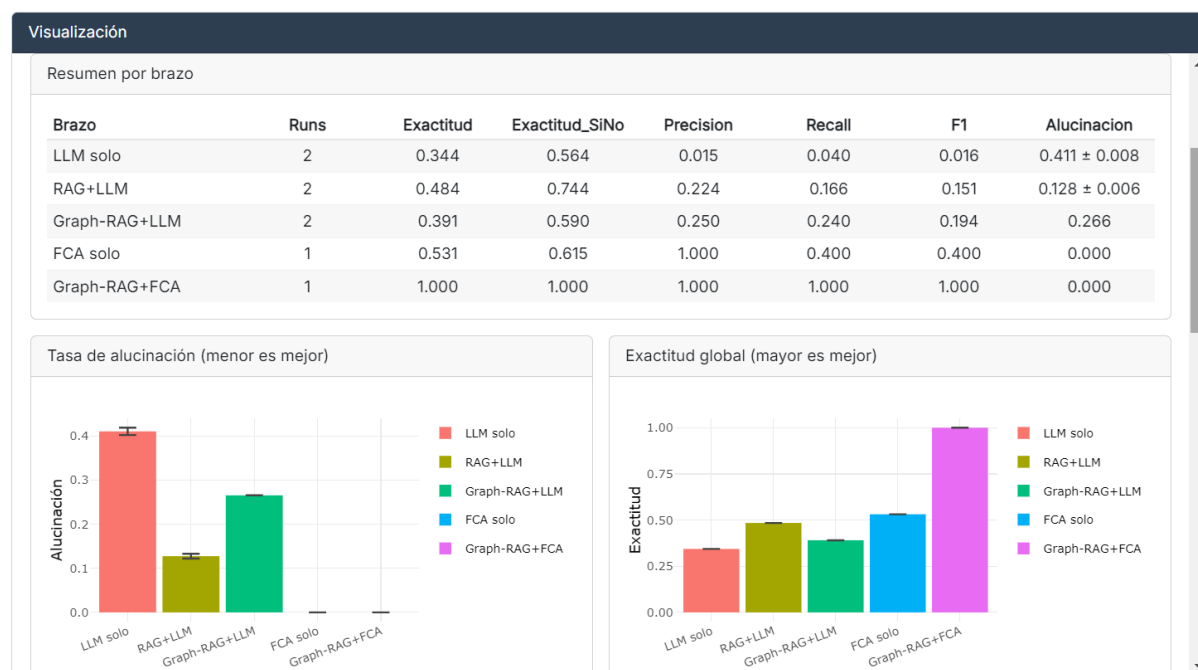


Figura 14 Evaluación – Comparativa de enfoques

6.3.4. Otros elementos transversales

Además de lo anterior, la interfaz incorpora una serie de elementos presentes de forma transversal:

- **Recorrido guiado:** un botón lanza un tutorial que recorre las distintas partes de la aplicación, especialmente útil para un usuario nuevo o durante una demostración.
- **Modo oscuro:** un conmutador permite alternar entre tema claro y oscuro.
- **Validación y realimentación:** la validación de los formularios es no intrusiva, y las operaciones largas muestran el progreso, mientras que las notificaciones de éxito o error aparecen de forma discreta.
- **Exportación:** es posible descargar tanto el CSV de tripletas generado en la extracción como los resultados de la evaluación en formato HTML o CSV.

6.4. Caso de uso: consulta clínica

Para ilustrar la utilidad de lo anteriormente expuesto, se plantea un escenario en el que un profesional desea explorar el conocimiento clínico de un conjunto de pacientes y razonar sobre sus riesgos.

En primer lugar, se carga el conocimiento, ya sea extrayéndolo de un informe clínico con el modelo de lenguaje, o bien subiendo el CSV de tripletas, y se construye el contexto formal. En este caso se emplea **datos_medicos.csv**

A continuación, sería posible realizar una **consulta** sobre un paciente concreto, por ejemplo qué condiciones tiene o qué le pasa. El asistente responde con la traza lógica del cierre deducido y con la resolución redactada, y el grafo enfoca simultáneamente al paciente y a sus atributos.

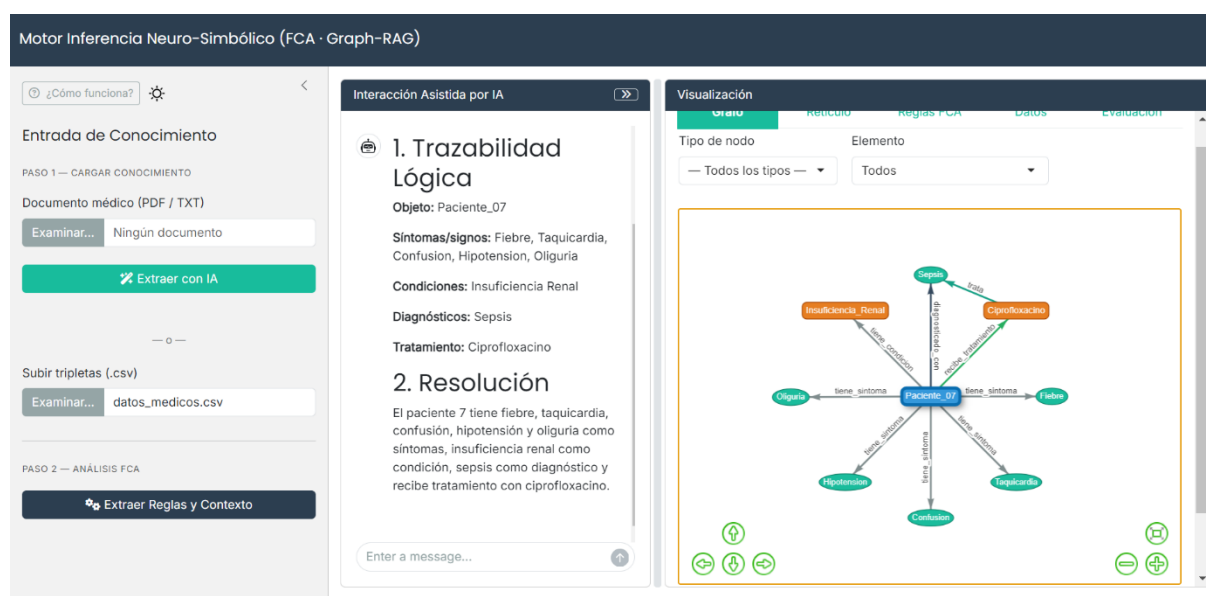


Figura 15 Consulta sobre un paciente, con la traza y enfoque del grafo

Más interesante resulta una **consulta hipotética**, del tipo "si un paciente tiene una determinada condición, ¿qué riesgos tendría / qué le pasaría / tendría o adquiriría algo?". En este caso el sistema aplica el cierre lógico y, sobre él, la propagación por el grafo de la relación de implicación, de modo que recupera los riesgos asociados aunque no estuvieran presentes de forma directa. El grafo resalta específicamente esas aristas de implicación, dejando ver la aportación de la capa Graph-RAG.

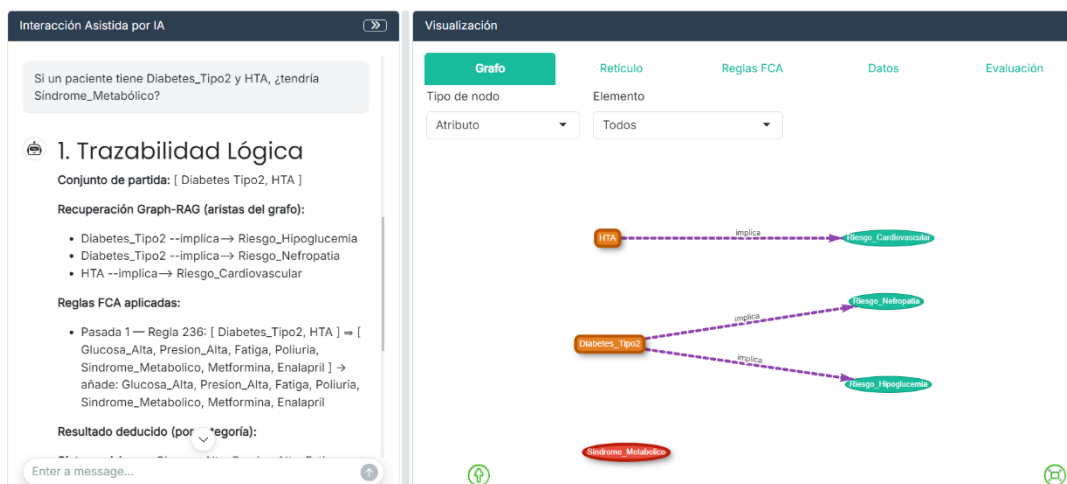


Figura 16 Consulta hipotética – Trazabilidad lógica

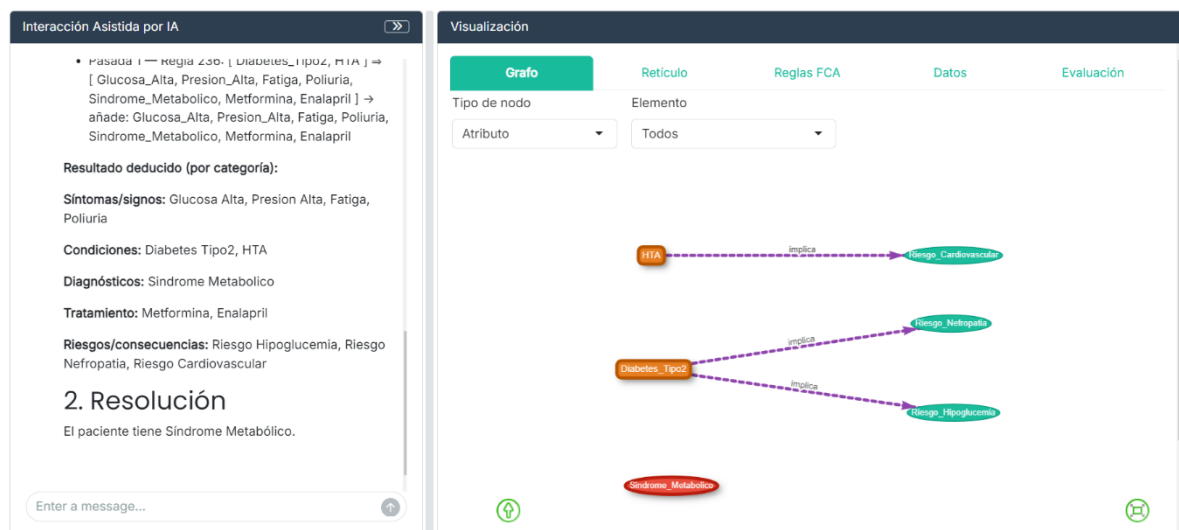


Figura 17 Consulta hipotética – Resolución

Por último, es posible plantear una **consulta inversa**, preguntando qué pacientes comparten un determinado atributo. Esto permite, por ejemplo, agrupar a todos los pacientes que presentan una misma condición.

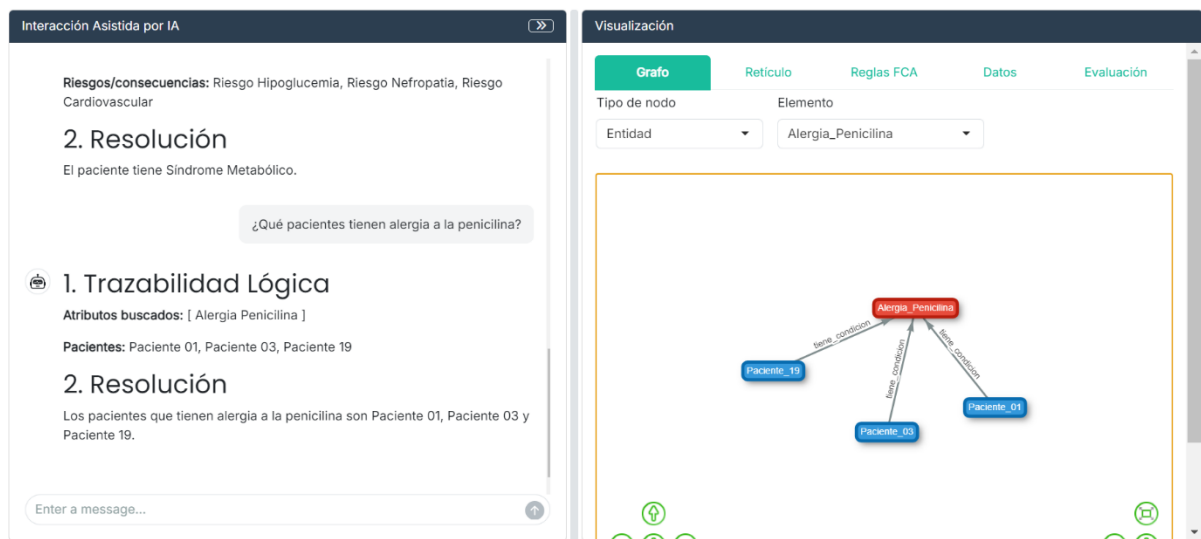


Figura 18 Consulta inversa

A modo de resumen, mediante la aplicación se ha conseguido:

- **Extraer conocimiento** clínico estructurado **a partir de texto y representarlo formalmente**.
- **Razonar de forma determinista sobre ese conocimiento**, con una traza lógica verificable y sin alucinación.
- **Recuperar mediante el grafo** los riesgos y consecuencias asociados a una condición.
- **Presentar todo el proceso de forma transparente e interactiva**, diferenciando en cada respuesta la parte deducida por el motor de la parte redactada por el modelo de lenguaje.

7

Validación y pruebas

8

Conclusiones y líneas futuras

Referencias

- [1] B. Ganter y R. Wille, Formal Concept Analysis: Mathematical Foundations. Springer, 1999. <https://link.springer.com/book/10.1007/978-3-642-59830-2>
- [2] Formal Concept Analysis: a Structural Framework for Variability Extraction and Analysis, arXiv:2508.06668, 2025. <https://arxiv.org/abs/2508.06668>
- [3] P. Cordero, M. Enciso, D. López-Rodríguez y Á. Mora, "fcaR, Formal Concept Analysis with R", The R Journal, vol. 14, n.º 1, 2022. <https://journal.r-project.org/articles/RJ-2022-014/>
- [4] D. López-Rodríguez, Á. Mora, J. Domínguez y A. Villalón, Package 'fcaR': Tools for Formal Concept Analysis. CRAN. <https://cran.r-project.org/package=fcaR>
- [5] X.-W. Yang et al., "Neuro-Symbolic Artificial Intelligence: Towards Improving the Reasoning Abilities of Large Language Models", IJCAI, 2025. <https://www.ijcai.org/proceedings/2025/1195>
- [6] A Neurosymbolic Approach to Natural Language Formalization and Verification, arXiv:2511.09008, 2025. <https://arxiv.org/abs/2511.09008>
- [7] M. Rani, B. K. Mishra y D. Thakker, Advancing Symbolic Integration in Large Language Models: Beyond Conventional Neurosymbolic AI, arXiv:2510.21425, 2025. <https://arxiv.org/abs/2510.21425>
- [8] Logically Consistent Language Models via Neuro-Symbolic Integration, OpenReview (ICLR 2025), arXiv:2409.13724. <https://openreview.net/forum?id=7PGluppo4k>
- [9] C. Qi et al., Large Language Models Meet Symbolic Provers for Logical Reasoning Evaluation, ICLR 2025, arXiv:2502.06563. <https://arxiv.org/abs/2502.06563>
- [10] Intermediate Languages Matter: Formal Languages and LLMs affect Neurosymbolic Reasoning, KG-NeSy Workshop (SEMANTiCS'25), CEUR-WS Vol-4064. [https://ceur-
ws.org/Vol-4064/kgnesy-paper3.pdf](https://ceur-ws.org/Vol-4064/kgnesy-paper3.pdf)

- [11] G. Aranda-Corral, A. Bundy, J. Borrego-Díaz y P. Chan, Grounding problem in Formal Concept Analysis by means of Large Language Models, CONCEPTS 2024. <https://concepts2024.uca.es/wp-content/uploads/2024/09/Gonzalo.pdf>
- [12] F. Boissier, M. Sen e I. Rychkova, Large Language Model and Formal Concept Analysis: a comparative study for Topic Modeling, arXiv:2602.01933, 2026. <https://arxiv.org/abs/2602.01933>
- [13] Evolution of Trustworthy AI: Formal Concept Analysis as a Mathematical Logic Foundation for Graph-RAG Architectures. [Sin URL pública verificable — revisa su procedencia antes de citarla]
- [14] C.-Y. Lin y J.-S. Jang, "Concept-Based RAG Models: A High-Accuracy Fact Retrieval Approach", FinNLP/FNP/LLMFinLegal Workshop, ACL Anthology, 2025. <https://aclanthology.org/2025.finnlp-1.8/>
- [15] Q. Zhang, S. Chen, Y. Bei, et al., A Survey of Graph Retrieval-Augmented Generation for Customized Large Language Models, arXiv:2501.13958, 2025. <https://arxiv.org/abs/2501.13958>
- [16] H. Han, Y. Wang, H. Shomer, K. Guo, et al., Retrieval-Augmented Generation with Graphs (GraphRAG), arXiv:2501.00309, 2024. <https://arxiv.org/abs/2501.00309>
- [17] Z. Zhu, T. Huang, K. Wang, J. Ye y X. Chen, Graph-based Approaches and Functionalities in Retrieval-Augmented Generation: A Comprehensive Survey, arXiv:2504.10499, 2025. <https://arxiv.org/abs/2504.10499>
- [18] A. J. Oche, A. G. Folashade, T. Ghosal y A. Biswas, A Systematic Review of Key Retrieval-Augmented Generation (RAG) Systems: Progress, Gaps, and Future Directions, arXiv:2507.18910, 2025. <https://arxiv.org/abs/2507.18910>
- [19] M. Li, S. Miao y P. Li, Simple is Effective: The Roles of Graphs and Large Language Models in Knowledge-Graph-based Retrieval-Augmented Generation, ICLR 2025, arXiv:2410.20724. <https://arxiv.org/abs/2410.20724>
- [20] J. Barry et al., "GraphRAG: Leveraging Graph-Based Efficiency to Minimize Hallucinations in LLM-Driven RAG for Finance Data", GenAIK Workshop, ACL Anthology, 2025. <https://aclanthology.org/2025.genaik-1.6/>
- [21] Y. Xiao et al., GraphRAG-Bench: Challenging Domain-Specific Reasoning for Evaluating Graph Retrieval-Augmented Generation, arXiv:2506.02404, 2025. <https://arxiv.org/abs/2506.02404>

- [22] F. Petruzzellis, C. Cornelio y P. Lio, Hierarchical Planning for Complex Tasks with Knowledge Graph-RAG and Symbolic Verification, ICML 2025, arXiv:2504.04578. <https://arxiv.org/abs/2504.04578>
- [23] Mitigating Hallucination in Large Language Models (LLMs): An Application-Oriented Survey on RAG, Reasoning, and Agentic Systems, arXiv:2510.24476, 2025. <https://arxiv.org/abs/2510.24476>
- [24] D. Anh-Hoang, V. Tran y L.-M. Nguyen, "Survey and analysis of hallucinations in large language models: attribution to prompting strategies or model behavior", Frontiers in Artificial Intelligence, 2025. <https://pmc.ncbi.nlm.nih.gov/articles/PMC12518350/>
- [25] Detecting Hallucinations in Graph Retrieval-Augmented Generation via Attention Patterns and Semantic Alignment, arXiv:2512.09148, 2025. <https://arxiv.org/abs/2512.09148>
- [26] Y. Dong, R. Mu, G. Jin, et al., Building Guardrails for Large Language Models, arXiv:2402.01822, 2024. <https://arxiv.org/abs/2402.01822>
- [27] RAG Makes Guardrails Unsafe? Investigating Robustness of Guardrails under RAG-style Contexts, arXiv:2510.05310, 2025. <https://arxiv.org/abs/2510.05310>
- [28] H. Wickham, J. Cheng, A. Jacobs, G. Aden-Buie y B. Schloerke, ellmer: Chat with Large Language Models. Paquete de R. <https://ellmer.tidyverse.org>
- [29] W. Chang, J. Cheng, J. J. Allaire, C. Sievert, B. Schloerke, Y. Xie, et al., shiny: Web Application Framework for R. <https://shiny.posit.co>
- [30] G. Csárdi y T. Nepusz, "The igraph software package for complex network research", InterJournal, Complex Systems, 1695, 2006. <https://igraph.org>
- [31] B. Thieurmel y T. Robert, visNetwork: Network Visualization using 'vis.js' Library. CRAN. <https://cran.r-project.org/package=visNetwork>
- [32] Ollama: Run large language models locally. <https://ollama.com>

Apéndice A.

Manual de Usuario



UNIVERSIDAD
DE MÁLAGA

| **uma.es**

E.T.S. DE INGENIERÍA INFORMÁTICA

E.T.S de Ingeniería Informática
Bulevar Louis Pasteur, 35
Campus de Teatinos
29071 Málaga